

# Time-Series Machine Learning: Placement and Interview Guide

---

This handbook develops time-series reasoning from data structure and statistical baselines to deep sequence models, state-space methods, and causal forecasting. Every chapter can be read independently; together they form a practical path for ML placements, AI-engineering interviews, research internships, and portfolio projects.

## Contents

1. Time Series Data
2. Trend
3. Seasonality
4. Stationarity
5. Moving Average
6. Exponential Smoothing
7. Train/Test Split for Time Series
8. Forecasting Metrics
9. ARIMA
10. SARIMA
11. Prophet
12. LSTM for Time Series
13. Temporal CNN
14. Transformer for Time Series
15. State Space Models
16. Kalman Filter
17. Causal Forecasting

**Notation.**  $y_t$  is the target at time  $t$ ;  $h$  is forecast horizon;  $L$  is lookback length;  $m$  is seasonal period;  $e_t = y_t - \hat{y}_t$  is forecast error. Information available after the forecast origin must never be used to construct a training feature.

---

## Time Series Data

---

### 1. Overview

Time-series data are observations indexed in time order. Unlike ordinary tabular rows, observations are usually dependent: today's demand is related to yesterday's demand, and this morning's sensor value constrains the next value. The order, spacing, and timestamp therefore carry predictive information.

Time series may be univariate ( $y_t$  only), multivariate (several jointly evolving variables), regularly sampled, irregularly sampled, continuous-valued, categorical events, or panels containing many related series. They drive demand and revenue forecasts, capacity planning, anomaly detection, predictive maintenance, finance, traffic, energy, weather, health monitoring, and observability.

## 2. Intuition

A shuffled photograph collection is still meaningful; a shuffled movie is not. A time series is closer to the movie. Its current frame depends on earlier frames, and a prediction must be made using only the frames already observed. For example, tomorrow's ice-cream sales may depend on recent sales, weekday, temperature forecast, holidays, and a long-term growth trend.

## 3. Prerequisites

- Python, NumPy, pandas, plotting, and basic SQL/time handling.
- Mean, variance, covariance, correlation, probability, and sampling.
- Regression, loss functions, overfitting, and feature engineering.
- Lags, rolling windows, autocorrelation, and temporal validation.
- For deep models: tensors, backpropagation, optimizers, and sequence batching.

## 4. Core Concepts

| Subtopic           | Meaning and importance                                            | Example                                   | Common interview angle                         |
|--------------------|-------------------------------------------------------------------|-------------------------------------------|------------------------------------------------|
| Timestamp/index    | Time associated with an observation; defines ordering and spacing | Hourly power readings                     | Why can timestamps not be discarded?           |
| Lag                | Earlier value $y_{\{t-k\}}$ used at time $t$                      | Yesterday's sales                         | How do lags create supervised samples?         |
| Autocorrelation    | Correlation between a series and a lagged copy                    | Weekly demand has high lag-7 correlation  | ACF versus ordinary correlation                |
| Exogenous variable | Known outside driver $x_t$                                        | Price, promotion, weather                 | Must it be known at forecast time?             |
| Forecast horizon   | Number of future steps requested                                  | Next 24 hours                             | Direct vs recursive multi-step output          |
| Frequency          | Intended interval between observations                            | 5 minutes, day, month                     | How do missing/duplicate timestamps affect it? |
| Panel series       | Many related entities indexed by time                             | Sales per store-SKU                       | Local versus global models                     |
| Event time         | When an event happened, distinct from ingestion time              | Payment timestamp vs arrival in warehouse | Point-in-time correctness and leakage          |

Important structure includes trend, seasonality, cycles, structural breaks, outliers, missing intervals, changing variance, and noise. A series can contain several simultaneously.

## 5. Algorithm / Working Process

1. Define target, entity, sampling frequency, forecast origin, horizon, and decision the forecast supports.

2. Sort by entity and event time; resolve duplicates and timezone/DST rules.
3. Reindex to the expected frequency so missing timestamps become visible.
4. Visualize levels, differences, rolling statistics, seasonal slices, and anomalies.
5. Create only point-in-time-valid lags, rolling summaries, calendar variables, and known-future covariates.
6. Split chronologically; fit preprocessing on training history only.
7. Establish naive baselines, train candidates, backtest over several origins, and inspect errors by horizon and segment.
8. Deploy with the same feature cutoff, then monitor data delay, drift, residual bias, and business impact.

Input is historical observations plus optional past/known-future covariates. Output may be a point forecast, quantiles, prediction interval, class, anomaly score, or latent state.

## 6. Mathematical Foundation

A general data-generating view is

$$y_t = f(y_{t-1}, \dots, y_{t-L}, x_t, x_{t-1}, \dots, s_t) + \epsilon_t$$

where  $s_t$  is a latent state and  $\epsilon_t$  is unpredictable noise. Autocovariance at lag  $k$  is

$$\begin{aligned} \gamma(k) &= \text{Cov}(y_t, y_{t-k}) \\ \rho(k) &= \gamma(k) / \gamma(0) \end{aligned}$$

The supervised-window representation is  $X_t = [y_{t-L}, \dots, y_{t-1}, \text{known features}]$ , target  $Y_t = [y_t, \dots, y_{t+h-1}]$ . This conversion creates overlapping, dependent samples; random row splitting would leak nearby information.

Missingness is often informative. Formally model both value and indicator  $M_t$ ; blindly imputing assumes missingness introduces no relevant signal. A probabilistic forecast estimates  $p(y_{t+1:t+h} | F_t)$ , where  $F_t$  is all information legitimately available at origin  $t$ .

## 7. Practical Implementation

```
import numpy as np
import pandas as pd

def make_supervised(series: pd.Series, lags=(1, 7, 14), horizon=1):
    """Create leakage-safe rows; pandas aligns each feature to its past."""
    s = series.sort_index().astype(float).rename("y")
    frame = pd.DataFrame({"y": s})
    for lag in lags:
        frame[f"lag_{lag}"] = s.shift(lag)
    # shift first, then roll: the current target never enters its own feature
    frame["mean_7"] = s.shift(1).rolling(7, min_periods=7).mean()
    frame["std_7"] = s.shift(1).rolling(7, min_periods=7).std()
    frame["day_of_week"] = frame.index.dayofweek
```

```

for step in range(horizon):
    frame[f"target_t+{step}"] = s.shift(-step)
return frame.dropna()

idx = pd.date_range("2025-01-01", periods=120, freq="D")
rng = np.random.default_rng(7)
sales = pd.Series(100 + 0.2*np.arange(120) + 8*np.sin(2*np.pi*np.arange(120)/7)
                  + rng.normal(0, 2, 120), index=idx)
data = make_supervised(sales, horizon=3)
cutoff = int(len(data) * 0.8)
train, test = data.iloc[:cutoff], data.iloc[cutoff:]
print(train.shape, test.shape, train.index.max() < test.index.min())

```

## 8. Code Explanation

`shift(lag)` aligns historical values with a later target. The rolling window is applied after `shift(1)`, preventing `yt` from entering a feature used to predict `yt`. Calendar values are known at prediction time. Future targets are shifted backward only to label rows, and incomplete boundary rows are removed. `iloc` preserves chronology in the split.

## 9. Training / Evaluation

- Split by timestamp and entity coverage, not random rows.
- Compare against last-value, seasonal-naive, mean, and drift baselines.
- Use rolling-origin backtesting and report metrics per horizon, entity, regime, and business slice.
- Scale using training statistics. Fit imputers, encoders, and feature selection inside each backtest fold.
- Diagnose overfitting as a widening train-to-future gap; tune lookback, regularization, model capacity, and feature count.
- Preserve a final untouched temporal test interval after model selection.

## 10. Complexity and Cost

Storage for  $n$  points and  $L$  lag features is  $O(nL)$  if materialized, but streaming features need only  $O(L)$  state per series. Sorting is commonly  $O(n \log n)$ ; rolling statistics can be  $O(n)$  with incremental updates. Classical models run comfortably on CPUs. Large panels and deep global models benefit from GPUs; serving cost depends on horizon, feature retrieval, and number of entities.

## 11. Common Use Cases

- Retail demand, inventory, revenue, and workforce planning.
- Energy load, renewable generation, and price forecasting.
- Sensor anomaly detection and remaining useful life.
- Traffic, ETA, network capacity, and cloud-resource prediction.
- Financial risk/volatility analysis and macroeconomic nowcasting.
- Patient vital monitoring and disease progression.

## 12. Common Mistakes

- Randomly shuffling rows or computing normalization over the full dataset.

- Using revised data, future weather, final inventory, or any feature unavailable at inference.
- Treating missing timestamps as zero without domain justification.
- Ignoring timezone, daylight-saving, duplicate, late-arriving, and irregular observations.
- Reporting one aggregate metric without a naive baseline or horizon breakdown.
- Confusing correlation with causation and predictability with intervention effect.

## 13. Edge Cases / Limitations

Cold-start entities have little history; intermittent demand contains many zeros; irregular event data may need continuous-time/event models; structural breaks make distant history harmful; censored demand records sales rather than true demand; feedback loops make predictions alter future observations. Rare extreme events and long horizons have irreducible uncertainty, so calibrated intervals and fallback policies matter.

## 14. Variations

| Variation              | What changes / when useful                | Importance                  |
|------------------------|-------------------------------------------|-----------------------------|
| Univariate             | Only target history                       | Placement baseline          |
| Multivariate           | Joint target/covariate dynamics           | Projects and research       |
| Panel/global           | One model shares learning across entities | Production interviews       |
| Irregular/event series | Unequal gaps or point processes           | Domain/research             |
| Hierarchical           | Forecasts must reconcile across levels    | Retail/business projects    |
| Probabilistic          | Quantiles/distributions instead of means  | High-value production skill |

## 15. Related Topics

Trend and seasonality describe systematic components; stationarity supports many statistical assumptions; moving averages and exponential smoothing are baselines; ARIMA models autocorrelation; state-space models represent hidden dynamics; LSTM, TCN, and Transformers learn nonlinear global patterns. Survival analysis concerns time-to-event, while causal forecasting estimates what would happen under interventions rather than merely what happens next.

## 16. Interview Questions

1. **Why is time-series ML different from tabular ML?** Rows are ordered and dependent, distributions may change, and only past information is valid at a forecast origin.
2. **What is a lag feature?** A prior observation such as  $y_{t-7}$  aligned as an input for predicting  $y_t$ .
3. **What is the horizon?** The number or duration of future steps predicted from one origin.
4. **Why not random split?** Adjacent observations and engineered windows leak future regimes into training.
5. **Known versus observed covariate?** Calendar and planned price can be known in the future; measured temperature is only observed unless separately forecast.
6. **Local versus global model?** A local model fits each series; a global model shares parameters across many series.
7. **How do you handle gaps?** Reindex, distinguish no-event from missing data, add missing indicators, and impute using training-safe/domain rules.

8. **How do you choose frequency?** Match decision cadence and signal dynamics while considering sparsity, latency, and aggregation loss.
9. **Point versus probabilistic forecast?** A point gives one value; a probabilistic forecast quantifies plausible outcomes and decision risk.
10. **What is the strongest leakage check?** Recreate every training row as it would have existed at its historical prediction timestamp.

## 17. Practice Tasks

- Write a leakage-safe window generator for multiple entities.
- Forecast hourly bike demand and compare hourly versus daily aggregation.
- Experiment with lag length and known-future weather forecasts.
- Debug a pipeline whose random-split score collapses in production.
- Extend point forecasts to 10th/50th/90th quantiles and test coverage.

## 18. Project Ideas

| Project                     | What it does                                   | Stack/data                        | Resume value                      |
|-----------------------------|------------------------------------------------|-----------------------------------|-----------------------------------|
| Store-SKU forecaster        | Global multi-horizon demand with backtests     | pandas, LightGBM/PyTorch; M5      | Panels, leakage, business metrics |
| Predictive maintenance      | Detects degradation and estimates failure risk | NumPy, sklearn; NASA C-MAPSS      | Sensors, windows, imbalance       |
| Energy operations dashboard | Load forecast with uncertainty and drift       | statsmodels/PyTorch, FastAPI; PJM | End-to-end monitoring             |

## 19. Quick Revision

- **Key idea:** order and information availability are part of the data.
- **Main formula:**  $p(y_{\text{future}} \mid \text{valid history } F_t)$ .
- **When to use:** any target indexed by time or affected by temporal dependence.
- **Metrics:** MAE/RMSE/MASE plus interval coverage and business cost.
- **Traps:** shuffled split, future features, wrong frequency, no baseline.
- **Interview one-liner:** "I define the forecast origin first, then make every feature and evaluation fold point-in-time correct."

## 20. Final Cheat Sheet

| Item            | Summary                                                                   |
|-----------------|---------------------------------------------------------------------------|
| Definition      | Ordered observations whose timestamps and dependence matter               |
| Input/output    | Past target/covariates → future values, distribution, or anomaly/state    |
| Main steps      | Define origin → clean index → engineer valid history → backtest → monitor |
| Hyperparameters | Frequency, lookback, horizon, lag set, model capacity                     |
| Metrics         | MAE, RMSE, WAPE/MASE, pinball loss, coverage                              |

| Item           | Summary                                                              |
|----------------|----------------------------------------------------------------------|
| Pros/cons      | Rich predictive structure / drift, leakage, and uncertainty are hard |
| Best use cases | Repeated operational decisions with historical temporal signal       |

## Trend

### 1. Overview

Trend is a persistent long-run change in the level of a series: growth, decline, saturation, or a piecewise change in direction. It is useful for capacity, revenue, adoption, and strategic planning and must be separated from seasonality and short-term noise. A trend may be deterministic (a fixed function of time) or stochastic (e.g., a random walk whose shocks have permanent effects).

### 2. Intuition

Think of climbing a hill while taking small steps up and down. The hill is trend; steps are local noise; repeated up-down patterns might be seasonality. A growing subscription product can have a rising trend even though weekend usage repeatedly falls.

### 3. Prerequisites

- Linear/polynomial regression, slopes, residuals, and extrapolation.
- Rolling averages, differencing, decomposition, stationarity.
- Change points, regularization, and temporal validation.

### 4. Core Concepts

| Subtopic            | Meaning / why it matters            | Example                   | Interview angle                |
|---------------------|-------------------------------------|---------------------------|--------------------------------|
| Linear trend        | Constant additive change per step   | +100 users/month          | Why extrapolation can fail     |
| Nonlinear trend     | Growth rate changes with level/time | Logistic adoption         | Polynomial instability         |
| Deterministic trend | $g(t)$ plus stationary residual     | Manufacturing improvement | Detrend by regression          |
| Stochastic trend    | Unit-root process; shocks persist   | Asset price random walk   | Difference rather than regress |
| Local trend         | Level/slope evolve over time        | Demand growth slows       | State-space formulation        |
| Structural break    | Abrupt slope/level change           | Policy or pandemic        | Backtesting across regimes     |

### 5. Algorithm / Working Process

1. Plot raw and log-scaled data with rolling summaries.
2. Compare linear, robust, piecewise, and smooth trend candidates.
3. Estimate trend only from data available at each fold.
4. Subtract it for residual modeling or include time explicitly in one joint model.
5. Forecast the trend, forecast residuals, then recombine.
6. Stress-test extrapolation and change-point sensitivity.

Input is  $(t, y_t)$  and optional change points/covariates; output is fitted trend  $\hat{g}(t)$  and detrended residual  $r_t = y_t - \hat{g}(t)$  (or division for multiplicative structure).

## 6. Mathematical Foundation

Additive decomposition:

$$y_t = g(t) + s_t + r_t, \quad g(t) = \beta_0 + \beta_1 t$$

Least squares chooses  $\beta$  to minimize  $\sum_t (y_t - \beta_0 - \beta_1 t)^2$ ;  $\beta_1 = \text{Cov}(t, y) / \text{Var}(t)$ . A multiplicative structure  $y_t = T_t S_t E_t$  becomes additive after logs. First differencing removes a linear deterministic trend and transforms a random walk:

$$\Delta y_t = y_t - y_{t-1}; \quad y_t = y_{t-1} + c + \epsilon_t \Rightarrow \Delta y_t = c + \epsilon_t$$

A piecewise-linear trend can be  $g(t) = \beta_0 + \beta_1 t + \sum_j \delta_j \max(0, t - c_j)$ , where each change point  $c_j$  changes the slope.

## 7. Practical Implementation

```
import numpy as np
from sklearn.linear_model import HuberRegressor

def fit_robust_trend(y, future_steps=12):
    """Robust linear trend; useful when isolated outliers should not tilt
    slope."""
    y = np.asarray(y, dtype=float)
    t = np.arange(len(y)).reshape(-1, 1)
    model = HuberRegressor().fit(t, y)
    fitted = model.predict(t)
    future_t = np.arange(len(y), len(y) + future_steps).reshape(-1, 1)
    return fitted, y - fitted, model.predict(future_t), model.coef_[0]

rng = np.random.default_rng(2)
y = 20 + 0.4*np.arange(80) + rng.normal(0, 2, 80)
y[30] += 30 # isolated outlier
trend, residual, forecast, slope = fit_robust_trend(y)
print(f"estimated slope={slope:.3f}, future={forecast[:3]}")
```

## 8. Code Explanation

Time is the sole regression feature. **HuberRegressor** behaves quadratically for small errors and approximately linearly for large errors, so one spike has less leverage than under ordinary least squares. The fitted values estimate in-sample trend; future integer indices extrapolate it. This is a baseline, not evidence the linear slope will persist.

## 9. Training / Evaluation

Fit trend within every temporal fold. Compare against drift ( $y_T + h(y_T - y_1)/(T-1)$ ), last-value, damped-trend, and no-trend models. Measure residual autocorrelation and bias by horizon. Regularize nonlinear bases and validate change-point flexibility; a flexible curve can interpolate history but extrapolate wildly. Prefer shorter, relevant history after a genuine regime break.

## 10. Complexity and Cost

Linear trend fitting is  $O(n)$  with sufficient statistics and  $O(1)$  prediction per horizon; robust iterative regression is roughly  $O(nI)$ . Splines/change-point searches cost more but remain CPU-friendly. Trend has tiny memory and serving cost.

## 11. Common Use Cases

- Long-term sales, user adoption, traffic, and capacity growth.
- Sensor degradation and remaining useful life.
- Inflation/population/economic trajectories.
- Detrending before ARMA spectral or anomaly analysis.

## 12. Common Mistakes

- Calling a temporary cycle a permanent trend.
- Fitting on full data before the split.
- Extrapolating high-degree polynomials.
- Using additive trend when variance/growth is proportional to level.
- Differencing repeatedly until signal is destroyed.
- Ignoring breaks, saturation, external drivers, and interval uncertainty.

## 13. Edge Cases / Limitations

Trends rarely continue forever. Physical bounds, market saturation, policy shocks, and feedback invalidate extrapolation. Endpoints are difficult for centered smoothers because future neighbors do not exist. A random walk can look like a smooth deterministic trend over short samples, and statistical tests have limited power.

## 14. Variations

| Variation         | Change and use                                       | Importance         |
|-------------------|------------------------------------------------------|--------------------|
| Log-linear        | Constant percentage growth                           | Placement/projects |
| Polynomial/spline | Smooth curvature; regularize and bound extrapolation | Projects           |

| Variation          | Change and use                         | Importance          |
|--------------------|----------------------------------------|---------------------|
| Piecewise linear   | Explicit slope changes                 | Prophet/interviews  |
| Damped trend       | Trend contribution decays with horizon | Strong baseline     |
| Local linear trend | Time-varying latent level and slope    | Research/production |
| Logistic trend     | Saturates at capacity                  | Product growth      |

## 15. Related Topics

Differencing and stationarity address stochastic trend; exponential smoothing estimates local level/slope; Prophet uses regularized piecewise trends; state-space models assign process noise to evolving trend; change-point detection finds breaks; seasonality is repeating rather than persistent directional movement.

## 16. Interview Questions

1. **What is trend?** Persistent long-run movement in series level.
2. **Trend versus seasonality?** Trend is directional; seasonality repeats at a known/estimable period.
3. **How remove linear trend?** Regress on time and subtract, or difference when a stochastic trend is plausible.
4. **Why is polynomial trend risky?** Small endpoint uncertainty grows dramatically outside the observed range.
5. **Deterministic versus stochastic?** Deterministic deviations are temporary around  $g(t)$ ; unit-root shocks permanently shift the path.
6. **When use logs?** Positive series with multiplicative growth/variance.
7. **What is damped trend?** A forecast whose slope effect geometrically decreases with horizon.
8. **How detect a break?** Residual monitoring, CUSUM/change-point tests, and fold-specific parameter instability.
9. **Why robust regression?** To stop isolated outliers from distorting level and slope.
10. **How validate extrapolation?** Simulate historical long-horizon forecasts across several origins and regimes.

## 17. Practice Tasks

- Estimate OLS and Huber slopes with injected outliers.
- Decompose airline passengers on raw and log scales.
- Compare linear, spline, damped, and logistic extrapolation.
- Debug a centered rolling-trend leakage bug.
- Add piecewise breakpoints selected by rolling validation.

## 18. Project Ideas

| Project                | Description                       | Stack/data                              | Resume value           |
|------------------------|-----------------------------------|-----------------------------------------|------------------------|
| SaaS growth forecaster | Saturating user/revenue scenarios | pandas, scipy; synthetic/public metrics | Business extrapolation |

| Project              | Description                             | Stack/data                   | Resume value           |
|----------------------|-----------------------------------------|------------------------------|------------------------|
| Sensor degradation   | Robust local trend predicts maintenance | sklearn/statsmodels; C-MAPSS | Change points/outliers |
| Macro trend explorer | Real-time-vintage trend comparison      | FRED, Plotly                 | Revisions and leakage  |

## 19. Quick Revision

- **Key idea:** long-run level movement separated from repeating/short noise.
- **Main formula:**  $y_t = \beta_0 + \beta_1 t + r_t$ .
- **Use:** growth, decline, degradation; **metrics:** horizon MAE/bias.
- **Trap:** unconstrained extrapolation through structural breaks.
- **One-liner:** "I distinguish deterministic from stochastic trend because it determines whether regression detrending or differencing is appropriate."

## 20. Final Cheat Sheet

| Item            | Summary                                                                 |
|-----------------|-------------------------------------------------------------------------|
| Definition      | Persistent direction in the expected level                              |
| Input/output    | Time and observations → trend/residual/future level                     |
| Steps           | Visualize → choose form → fold-wise fit → residual forecast → recombine |
| Hyperparameters | Window, damping, knots/change points, robustness                        |
| Metrics         | MAE/RMSE, bias, residual ACF, long-horizon stability                    |
| Pros/cons       | Interpretable baseline / fragile extrapolation                          |
| Best use        | Stable growth/decline or preprocessing                                  |

## Seasonality

### 1. Overview

Seasonality is a systematic pattern repeating at a fixed or calendar-linked period: hourly, daily, weekly, annual, holiday, or business-cycle positions. Modeling it improves accuracy, staffing, inventory, energy balancing, and anomaly thresholds. A series may have multiple and changing seasonalities, such as intraday and weekly traffic.

### 2. Intuition

A coffee shop is busy each weekday morning and quiet each night. The recurring within-day shape is daily seasonality; lower weekend activity is weekly seasonality. "Same time last week" often beats a sophisticated model because it directly respects this pattern.

### 3. Prerequisites

- Period/frequency, lag, autocorrelation, Fourier sine/cosine functions.
- Additive/multiplicative decomposition, trend, residuals.
- Calendar engineering, temporal validation, categorical encoding.

### 4. Core Concepts

| Subtopic             | Meaning / importance         | Example                         | Interview angle              |
|----------------------|------------------------------|---------------------------------|------------------------------|
| Seasonal period $m$  | Steps per repetition         | $m=7$ for daily weekly pattern  | Frequency versus period      |
| Additive             | Seasonal amplitude constant  | $\pm 100$ units each week       | When appropriate?            |
| Multiplicative       | Amplitude scales with level  | December is 1.4 $\times$ normal | Log transform                |
| Multiple seasonality | Several periods coexist      | Hourly data: 24 and 168         | Why SARIMA may struggle      |
| Calendar effect      | Position depends on calendar | Easter, month end               | Fixed period is insufficient |
| Seasonal drift       | Pattern changes over time    | New weekend behavior            | Regularize/recent weighting  |

### 5. Algorithm / Working Process

1. Establish frequency and candidate periods using domain knowledge.
2. Plot seasonal subseries (hour by hour, weekday by weekday) and ACF at seasonal lags.
3. Remove/allow trend so it does not masquerade as low-frequency seasonality.
4. Estimate seasonal indices, Fourier terms, seasonal lags, or model states.
5. Forecast future seasonal positions from their phase/calendar.
6. Backtest against seasonal-naïve forecasts at all relevant periods.

Input is timestamps and values; output is a seasonal component  $s_t$ , adjusted series, or direct forecast.

### 6. Mathematical Foundation

Additive and multiplicative forms are

$$y_t = T_t + S_t + R_t$$

$$y_t = T_t * S_t * R_t, \text{ or } \log(y_t) = \log(T_t) + \log(S_t) + \log(R_t)$$

For identifiability, additive indices often satisfy  $\sum_{j=1}^m S_j = 0$ ; multiplicative indices average to one. Seasonal differencing is  $\Delta_m y_t = y_t - y_{t-m}$ . Fourier regression represents smooth seasonality:

$$S_t = \sum_{k=1}^K [a_k \sin(2\pi k t/m) + b_k \cos(2\pi k t/m)]$$

Increasing  $K$  represents sharper shapes but increases overfitting. Seasonal-naive forecasting is  $\hat{y}_{T+h} = y_{T+h-Km}$ , choosing the latest observed matching phase.

## 7. Practical Implementation

```
import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge

def fourier_features(t, period, order):
    t = np.asarray(t)
    return np.column_stack([
        fn(2*np.pi*k*t/period)
        for k in range(1, order + 1) for fn in (np.sin, np.cos)
    ])

n, horizon = 365, 30
t = np.arange(n)
rng = np.random.default_rng(4)
y = 50 + 0.03*t + 7*np.sin(2*np.pi*t/7) + rng.normal(0, 1.5, n)
X = np.column_stack([t, fourier_features(t, period=7, order=3)])
model = Ridge(alpha=1.0).fit(X, y)
future_t = np.arange(n, n+horizon)
future_X = np.column_stack([future_t, fourier_features(future_t, 7, 3)])
forecast = model.predict(future_X)
print(forecast[:5])
```

## 8. Code Explanation

Each harmonic contributes sine and cosine columns, letting phase and amplitude be learned by linear regression. The time column models a basic trend. Three harmonics can express a non-sinusoidal weekly shape without seven unrelated dummy parameters. Ridge regularization stabilizes correlated components. Future features are deterministic because time phase is known.

## 9. Training / Evaluation

Choose periods from operations, not only a periodogram. Evaluate unseen complete seasonal cycles and report performance by phase (weekday/hour/holiday). Tune Fourier order on rolling folds. Compare seasonal-naive, calendar dummy, Fourier, and state-space models. Evaluate rare holidays over multiple years if possible. Diagnose residual ACF at  $m, 2m, \dots$ ; remaining peaks imply missed seasonality.

## 10. Complexity and Cost

Seasonal lookup is  $O(m)$  memory and  $O(1)$  prediction. Fourier regression with  $2K$  terms is about  $O(nK^2 + K^3)$  for a standard solver and cheap for small  $K$ . Seasonal models are CPU-friendly; high-cardinality panels and

neural models dominate cost, not seasonal encoding.

## 11. Common Use Cases

- Retail weekly/annual demand and holiday peaks.
- Intraday/weekly electricity, traffic, and cloud usage.
- Staffing for calls, hospitals, restaurants, and delivery.
- Seasonal anomaly baselines and climate/agriculture analysis.

## 12. Common Mistakes

- Setting `m` incorrectly (e.g., 12 for daily data without meaning).
- Confusing seasonality with any repeated-looking noise or longer economic cycles.
- Using future realized covariates while creating calendar features.
- Applying both seasonal differencing and rich seasonal features until over-differenced.
- Ignoring leap years, DST, movable holidays, closures, and trading calendars.
- Declaring success without beating seasonal-naïve.

## 13. Edge Cases / Limitations

Very long periods require multiple cycles for estimation. Calendar periods are not always constant in steps. Seasonal phase/amplitude can evolve after behavior changes. Missing blocks corrupt seasonal averages. Intermittent demand has weak conventional indices. Promotions correlated with holidays confound intrinsic seasonality with causal effects.

## 14. Variations

| Variation             | Change/use                                   | Importance            |
|-----------------------|----------------------------------------------|-----------------------|
| Seasonal dummies      | One parameter per phase                      | Basic interviews      |
| Fourier terms         | Smooth, compact long-period pattern          | Prophet/projects      |
| STL/MSTL              | Locally changing one/multiple seasonalities  | Strong practical tool |
| Seasonal differencing | Removes seasonal unit root                   | ARIMA interviews      |
| TBATS                 | Multiple/complex seasonality and Box-Cox     | Advanced classical    |
| Learned embeddings    | Share calendar effects in deep/global models | DL projects           |

## 15. Related Topics

SARIMA uses seasonal AR/MA/differencing; Holt-Winters updates seasonal states; Prophet combines Fourier seasonality with holidays; TCNs and Transformers can learn long seasonal dependencies; spectral analysis finds frequencies; trend decomposition prevents trend/season confusion.

## 16. Interview Questions

1. **Define seasonality.** A systematic pattern recurring at a fixed or calendar-linked phase.

2. **How identify it?** Domain knowledge, seasonal plots, ACF peaks, periodogram, and out-of-sample validation.
3. **Additive versus multiplicative?** Constant absolute amplitude versus amplitude proportional to level.
4. **What is seasonal differencing?**  $y_t - y_{t-m}$  to remove recurring seasonal level/unit root.
5. **Why Fourier terms?** They encode smooth long-period cycles with few parameters and extrapolate phase.
6. **How choose Fourier order?** Rolling validation; higher order adds sharper detail and variance.
7. **Seasonality versus cycle?** Seasonality has predictable calendar/fixed timing; cycles have variable duration.
8. **Multiple seasonalities?** Include multiple period bases/states or use MSTL/TBATS/global models.
9. **Best baseline?** Latest value from the corresponding seasonal phase.
10. **How test missed seasonality?** Inspect residual seasonal plots and ACF at seasonal multiples.

## 17. Practice Tasks

- Implement seasonal-naive prediction for arbitrary horizon.
- Compare weekday dummies with Fourier features on bike demand.
- Test additive versus log/multiplicative decomposition.
- Debug a wrong `m` and daylight-saving duplicate-hour issue.
- Extend a weekly model with holiday windows and changing amplitude.

## 18. Project Ideas

| Project              | Description                                           | Stack/data            | Resume value         |
|----------------------|-------------------------------------------------------|-----------------------|----------------------|
| Multi-seasonal load  | Forecast hourly load with daily/weekly/yearly effects | pandas, MSTL; PJM     | Multiple periods     |
| Holiday demand lab   | Quantifies recurring and movable holiday effects      | statsmodels; M5       | Calendar correctness |
| Seasonal anomaly API | Phase-aware alerts for service traffic                | sklearn, FastAPI; NAB | Production baseline  |

## 19. Quick Revision

- **Key idea:** predictable pattern indexed by recurring phase.
- **Formula:**  $S_t = \sum_k a_k \sin(2\pi kt/m) + b_k \cos(2\pi kt/m)$ .
- **Use:** calendar-driven operations; **metric:** fold/horizon error vs seasonal-naive.
- **Trap:** wrong period or future holiday/covariate leakage.
- **One-liner:** "Seasonality repeats with a stable phase; I validate its period and always benchmark against seasonal-naive."

## 20. Final Cheat Sheet

| Item         | Summary                                            |
|--------------|----------------------------------------------------|
| Definition   | Recurring time/calendar pattern                    |
| Input/output | Timestamp/history → seasonal component or forecast |

| Item            | Summary                                                              |
|-----------------|----------------------------------------------------------------------|
| Steps           | Pick periods → visualize/ACF → encode → backtest → inspect residuals |
| Hyperparameters | Period $m$ , Fourier order, seasonal window/damping                  |
| Metrics         | MAE/RMSE/MASE by phase/horizon                                       |
| Pros/cons       | Strong predictable signal / shifts and calendars complicate it       |
| Best use        | Retail, traffic, energy, staffing, anomaly baselines                 |

## Stationarity

### 1. Overview

Stationarity means the probabilistic behavior of a process does not change with time. Strict stationarity requires every joint distribution to be invariant to a time shift. Weak (covariance) stationarity—the usual modeling assumption—requires constant finite mean and variance and autocovariance depending only on lag. It matters because ARMA estimation, ACF interpretation, and many theoretical guarantees assume stable relationships.

### 2. Intuition

Imagine repeatedly taking a short clip from a machine sensor. If clips from January and June have similar level, spread, and lag relationships, the process is plausibly stationary. If the machine steadily wears down, the distribution changes and a rule learned in January may fail in June.

### 3. Prerequisites

- Expectation, variance, covariance, autocorrelation.
- Trend, seasonality, differencing, transformations.
- Hypothesis testing, p-values, unit roots, residual diagnostics.

### 4. Core Concepts

| Subtopic                | Meaning / importance                              | Example                  | Interview angle              |
|-------------------------|---------------------------------------------------|--------------------------|------------------------------|
| Strict stationarity     | Full joint distribution shift-invariant           | Idealized process        | Stronger than usually needed |
| Weak stationarity       | Constant first two moments, lag-only covariance   | Stable AR(1)             | Conditions and assumptions   |
| Unit root               | Root at one; shocks persist                       | Random walk              | ADF null hypothesis          |
| Difference stationarity | Differencing yields stationary process            | ARIMA $d=1$              | Over-differencing risk       |
| Trend stationarity      | Residual around deterministic trend is stationary | Linear growth + AR noise | Detrend vs difference        |

| Subtopic         | Meaning / importance        | Example            | Interview angle                    |
|------------------|-----------------------------|--------------------|------------------------------------|
| Structural break | Parameters change at a date | New pricing regime | Tests can mistake it for unit root |

## 5. Algorithm / Working Process

1. Plot levels, rolling mean/variance, seasonal slices, and ACF.
2. Use domain/regime knowledge and test for breaks.
3. Apply ADF (null: unit root) and optionally KPSS (null: stationary) as supporting evidence.
4. Transform variance (log/Box-Cox), detrend, and/or difference minimally.
5. Recheck plots/tests and fit a stationary residual model.
6. Invert transformations/differences carefully during forecasting.

Input is an ordered series; output is a diagnostic judgment and possibly a transformed stationary series.

## 6. Mathematical Foundation

Weak stationarity requires

$$\begin{aligned}
 E[y_t] &= \mu \\
 \text{Var}(y_t) &= \sigma^2 < \text{infinity} \\
 \text{Cov}(y_t, y_{t-k}) &= \gamma(k), \text{ independent of } t
 \end{aligned}$$

For  $y_t = \phi y_{t-1} + \epsilon_t$ , an AR(1) is stationary when  $|\phi| < 1$ ; then  $\text{Var}(y_t) = \sigma_e^2 / (1 - \phi^2)$  and  $\rho(k) = \phi^k$ . If  $\phi = 1$ , it is a random walk and variance grows with time. ADF estimates a form such as

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_i \delta_i \Delta y_{t-i} + \epsilon_t$$

and tests the unit-root null ( $\gamma = 0$ ) against stationarity ( $\gamma < 0$ ). KPSS reverses the logic: its null is level/trend stationarity. Tests do not “prove” stationarity; sample size, lag choice, breaks, and deterministic terms affect power.

## 7. Practical Implementation

```

import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import adfuller, kpss

def stationarity_report(values):
    y = pd.Series(values).dropna().astype(float)
    adf_stat, adf_p, *_ = adfuller(y, autolag="AIC")
    kpss_stat, kpss_p, *_ = kpss(y, regression="c", nlags="auto")
    return {"adf_p(unit-root null)": adf_p,

```

```

    "kpss_p(stationary null)": kpss_p,
    "mean": y.mean(), "variance": y.var()}

rng = np.random.default_rng(10)
random_walk = pd.Series(rng.normal(size=500)).cumsum()
print("level:", stationarity_report(random_walk))
print("difference:", stationarity_report(random_walk.diff()))

```

## 8. Code Explanation

The random walk accumulates innovations, so shocks persist and the level is nonstationary. First differencing approximately recovers the innovations. A small ADF p-value rejects its unit-root null; a large KPSS p-value fails to reject stationarity. Use both with visual and domain evidence rather than treating p-values as an automatic pipeline.

## 9. Training / Evaluation

Stationarity is a modeling diagnostic, not a forecast metric. Select transformation and difference orders inside rolling folds, then evaluate forecasts after inverse transformation. Residuals should have stable mean/variance and little autocorrelation; use ACF and Ljung-Box alongside error metrics. Over-differencing produces excessive negative lag-1 correlation and widens long-horizon uncertainty.

## 10. Complexity and Cost

Differencing and rolling diagnostics are  $O(n)$ . ADF regression cost depends on lag count and is small for ordinary datasets; memory is  $O(n)$ . CPU is sufficient. The real cost is losing signal or incorrectly inverting transformations, not computation.

## 11. Common Use Cases

- Preparing data for ARMA/ARIMA and spectral analysis.
- Identifying persistent shocks in economic and financial series.
- Modeling stable residuals after trend/season removal.
- Monitoring process stability and detecting regime change.

## 12. Common Mistakes

- Saying stationary means “constant values” rather than stable distribution.
- Declaring stationarity from one test/p-value.
- Ignoring whether ADF includes intercept/trend and the selected lag order.
- Differencing seasonal and ordinary components unnecessarily.
- Testing the full dataset and using the decision before a historical split.
- Assuming neural networks require stationarity or are immune to drift.

## 13. Edge Cases / Limitations

Short samples give low test power. Structural breaks, nonlinear trends, seasonal roots, changing variance, and long memory violate simple assumptions. Locally stationary processes may be stable only in short windows. A

stationary process may still have extreme heavy tails; second-order stationarity may not exist when variance is infinite.

## 14. Variations

| Variation             | Change/use                                       | Importance                 |
|-----------------------|--------------------------------------------------|----------------------------|
| Trend stationary      | Remove deterministic $g(t)$                      | Core interview distinction |
| Difference stationary | Difference unit-root process                     | ARIMA core                 |
| Seasonal stationarity | Remove seasonal roots with $\Delta_m$            | SARIMA                     |
| Variance stationary   | Log/Box-Cox stabilizes spread                    | Practical                  |
| Local stationarity    | Parameters slowly vary                           | Research/production        |
| Cointegration         | Nonstationary series have stationary combination | Econometrics/advanced      |

## 15. Related Topics

ARIMA's  $d$  targets integration; SARIMA's  $D$  targets seasonal roots; trend and seasonality cause common nonstationarity; cointegration permits meaningful regression among certain nonstationary series; state-space models explicitly allow evolving parameters; concept drift is the ML production analogue of changing distributions.

## 16. Interview Questions

1. **Define weak stationarity.** Constant mean/variance and autocovariance determined only by lag.
2. **Strict versus weak?** Strict concerns all joint distributions; weak only first two moments.
3. **Is a random walk stationary?** No; shocks persist and its variance grows with time.
4. **ADF null?** The series has a unit root.
5. **KPSS null?** The series is level- or trend-stationary, depending on specification.
6. **Why use both?** Their opposite nulls help distinguish evidence, though neither replaces judgment.
7. **How make a series stationary?** Detrend, difference, seasonal-difference, transform variance, or model regimes.
8. **What is over-differencing?** Removing more integration than exists, adding noise and negative autocorrelation.
9. **Do all forecasting models require it?** No, but stable distributions help learning; ARMA theory specifically relies on it.
10. **Stationarity versus white noise?** White noise is stationary and uncorrelated; stationary processes may have predictable autocorrelation.

## 17. Practice Tasks

- Simulate AR(1) at  $\phi = .7, .99, 1.0$  and compare ACF/tests.
- Decide trend/difference stationarity for macro data.
- Study how a structural break changes ADF conclusions.
- Debug double differencing and forecast reconstruction.
- Extend diagnostics with KPSS, Ljung-Box, and rolling parameter estimates.

## 18. Project Ideas

| Project                   | Description                                | Stack/data                       | Resume value      |
|---------------------------|--------------------------------------------|----------------------------------|-------------------|
| Unit-root laboratory      | Simulations explain test power and breaks  | NumPy, statsmodels               | Statistical depth |
| Regime monitor            | Detects changing mean/variance online      | scipy, River; server metrics     | Production drift  |
| Cointegration pairs study | Tests stable spreads with honest backtests | pandas, statsmodels; market data | Advanced caution  |

## 19. Quick Revision

- **Key idea:** distributional properties do not depend on absolute time.
- **Formula:**  $E[y_t] = \mu$ ,  $Cov(y_t, y_{t-k}) = \gamma(k)$ .
- **Use:** ARMA assumptions and stable residual modeling.
- **Trap:** blindly equating ADF p-value with truth.
- **One-liner:** "Weak stationarity stabilizes moments and lag relationships; I combine plots, ADF/KPSS, break checks, and forecast validation."

## 20. Final Cheat Sheet

| Item            | Summary                                                                  |
|-----------------|--------------------------------------------------------------------------|
| Definition      | Time-invariant mean/variance and lag covariance (weak form)              |
| Input/output    | Series → diagnosis/transformation                                        |
| Steps           | Plot → test → transform minimally → retest → model/invert                |
| Hyperparameters | ADF lags, deterministic term, $d$ , $D$ , transform                      |
| Metrics         | ADF/KPSS evidence, residual ACF/Ljung-Box, forecast error                |
| Pros/cons       | Enables stable inference / tests are fragile to breaks and small samples |
| Best use        | Statistical modeling and process diagnostics                             |

## Moving Average

### 1. Overview

"Moving average" has two related meanings. A rolling/moving-average smoother replaces a value by an average over a sliding window to expose local level and reduce noise. In ARMA models, an MA(q) process instead expresses  $y_t$  using current and past random shocks. Interviewers often test this distinction. Rolling averages support smoothing, feature engineering, monitoring, decomposition, and simple forecasts; MA error models capture short-lived shock dependence.

### 2. Intuition

A seven-day average asks, “What has the typical value been over the latest week?” One unusual day contributes only one-seventh and disappears when it leaves the window. An MA(1) stochastic model asks a different question: “Does today contain both a new surprise and part of yesterday’s surprise?”

### 3. Prerequisites

- Mean, convolution, lag, window alignment, and boundary handling.
- Autocorrelation, white noise, forecast error, stationarity.
- Leakage-safe feature engineering and chronological evaluation.

### 4. Core Concepts

| Subtopic                    | Meaning / why it matters                   | Example                     | Interview angle                             |
|-----------------------------|--------------------------------------------|-----------------------------|---------------------------------------------|
| Simple moving average (SMA) | Equal weights over last $w$ values         | Seven-day demand mean       | Lag versus smoothness tradeoff              |
| Weighted MA                 | Recent/important points get larger weights | Triangular weights          | Weights sum to one                          |
| Centered MA                 | Uses neighbors before and after $t$        | Historical trend extraction | Cannot be used online without leakage       |
| Trailing MA                 | Uses only values at/before origin          | Monitoring feature          | Shift needed when predicting same timestamp |
| MA(q) process               | Linear combination of innovations          | One-day shock carryover     | PACF/ACF signature, invertibility           |
| Rolling statistic           | Window mean, std, min, quantile            | Volatility feature          | Efficient incremental computation           |

### 5. Algorithm / Working Process

For a trailing smoother: choose window  $w$ ; at each forecast origin take the latest  $w$  valid observations; optionally weight them; compute the mean; use it as level, feature, or next forecast; update by adding the newest and removing the oldest value. For an MA(q) stochastic model, estimate coefficients from a stationary series, infer unobserved innovations, forecast future innovations as zero, and retain only the effect of already estimated shocks.

Input is a series and window/order. Output is a smoothed series, rolling features, or future predictions.

### 6. Mathematical Foundation

Simple and weighted moving averages are

$$\begin{aligned} \text{SMA}_t(w) &= (1/w) \sum_{i=0}^{w-1} y_{t-i} \\ \text{WMA}_t &= \sum_{i=0}^{w-1} a_i y_{t-i}, \quad \sum_i a_i = 1 \end{aligned}$$

For one-step prediction of  $y_t$ , a safe feature is  $(1/w) \sum_{i=1}^w y_{t-i}$ , excluding  $y_t$ . Averaging independent noise reduces its variance from  $\sigma^2$  to  $\sigma^2/w$ , but correlated noise reduces less. The MA( $q$ ) process is

$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q}$$

It is stationary by construction. Its ACF is zero after lag  $q$  theoretically; invertibility requires roots of  $1 + \theta_1 z + \dots + \theta_q z^q = 0$  outside the unit circle, ensuring a unique stable innovation representation.

## 7. Practical Implementation

```
from collections import deque
import pandas as pd

class OnlineMovingAverage:
    def __init__(self, window):
        if window < 1:
            raise ValueError("window must be positive")
        self.values, self.total = deque(maxlen=window), 0.0

    def update(self, value):
        if len(self.values) == self.values.maxlen:
            self.total -= self.values[0]
        self.values.append(float(value))
        self.total += float(value)
        return self.total / len(self.values)

s = pd.Series([10, 12, 11, 15, 14, 16])
# Safe feature for predicting row t: exclude row t before rolling.
rolling_feature = s.shift(1).rolling(window=3, min_periods=3).mean()
online = OnlineMovingAverage(3)
print([online.update(x) for x in s])
print(rolling_feature.tolist())
```

## 8. Code Explanation

The deque stores at most  $w$  observations. Maintaining a running sum makes each update  $O(1)$  rather than repeatedly summing  $w$  values. Pandas `shift(1)` makes the batch feature point-in-time safe. Early online estimates use fewer values; the batch feature instead requires a full window, which is a deliberate cold-start policy.

## 9. Training / Evaluation

The smoother has no conventional training; select window and weights using rolling-origin validation. Short windows react quickly but pass noise; long windows reduce noise but lag turning points. Compare with last-value and exponential smoothing. For MA( $q$ ), fit maximum likelihood on stationary training folds, inspect residual ACF/Ljung-Box, and select  $q$  with AIC/BIC plus out-of-sample error.

## 10. Complexity and Cost

Naive rolling computation is  $O(nw)$ ; cumulative/running-sum implementations are  $O(n)$  time and  $O(w)$  streaming memory. MA(q) likelihood typically requires iterative  $O(nq)$ -scale filtering per iteration. Both are CPU-friendly and inexpensive to serve.

## 11. Common Use Cases

- Noise reduction and dashboard trend lines.
- Rolling demand/volatility features for ML.
- Quality-control thresholds and sensor preprocessing.
- Baseline forecasts for stable local level.
- Modeling finite-lived error shocks within ARMA/ARIMA.

## 12. Common Mistakes

- Using a centered window or unshifted target-derived rolling feature in prediction.
- Ignoring boundary behavior and missing observations.
- Choosing window solely for a visually smooth plot.
- Treating a smoother's correlated residuals as independent.
- Confusing rolling SMA with the MA component of ARIMA.
- Using a long window across regime breaks.

## 13. Edge Cases / Limitations

Moving averages lag sudden turns, blur peaks, and handle trend/seasonality poorly unless the window matches the structure. Outliers contaminate  $w$  consecutive outputs; rolling median may be better. Irregular timestamps need time-based rather than row-count windows. MA(q) innovations are latent, so estimation is nonlinear despite its linear expression.

## 14. Variations

| Variation                  | What changes / when useful                          | Importance     |
|----------------------------|-----------------------------------------------------|----------------|
| Cumulative average         | Window includes all history; stable stationary mean | Basics         |
| Weighted/triangular        | Custom recency emphasis                             | Interviews     |
| Rolling median             | Robust to spikes                                    | Practical      |
| Exponential moving average | Infinite window with decaying weights               | Core practical |
| Seasonal moving average    | Window aligned to seasonal phases                   | Decomposition  |
| MA(q) error model          | Models latent shocks, not observed-value smoothing  | ARIMA core     |

## 15. Related Topics

Exponential smoothing removes the hard cutoff; convolution and FIR filters generalize weighted windows; TCNs learn filters; ARIMA combines autoregressive and innovation moving-average terms; STL uses smoothers for decomposition; rolling-window validation is unrelated despite similar terminology.

## 16. Interview Questions

1. **What is SMA?** Equal-weight mean of the latest  $w$  observations.
2. **Why does it smooth?** Independent zero-mean noise partially cancels under averaging.
3. **Main tradeoff?** Larger  $w$  lowers variance but increases lag/bias at changes.
4. **Centered versus trailing?** Centered uses future neighbors; trailing is usable online.
5. **How prevent feature leakage?** Shift the target before applying a rolling operator.
6. **SMA versus EMA?** SMA has a finite equal-weight window; EMA has infinite geometrically decaying weights.
7. **What is MA(q)?** A stochastic process driven by current and  $q$  prior innovations.
8. **ACF signature of MA(q)?** It cuts off after lag  $q$  in the population.
9. **Why invertibility?** It gives a unique stable mapping between observations and innovations.
10. **Efficient online update?** Maintain a queue and running sum: add new, subtract expired.

## 17. Practice Tasks

- Implement  $O(1)$  rolling mean and variance.
- Compare SMA windows on noisy seasonal demand.
- Demonstrate leakage from an unshifted rolling feature.
- Simulate MA(2) and examine sample ACF.
- Extend the online class to missing-value and time-window policies.

## 18. Project Ideas

| Project                  | Description                             | Stack/data                         | Resume value           |
|--------------------------|-----------------------------------------|------------------------------------|------------------------|
| Streaming KPI smoother   | Online mean/median with alerts          | Python, FastAPI; synthetic metrics | Stateful serving       |
| Signal filter comparison | SMA/EMA/Savitzky-Golay under noise      | NumPy, scipy; sensor data          | Bias/noise analysis    |
| MA diagnostics lab       | Simulate/recover finite shock processes | statsmodels                        | Statistical interviews |

## 19. Quick Revision

- **Key idea:** average a local window; distinguish it from MA( $q$ ) innovations.
- **Formula:**  $SMA_t = (1/w) \sum_{i=0}^{w-1} y_{t-i}$ .
- **Use:** smoothing, rolling features, simple stable-level forecast.
- **Trap:** unshifted/centered windows leak.
- **One-liner:** "A rolling moving average smooths observed values; ARIMA's MA term models past forecast shocks."

## 20. Final Cheat Sheet

| Item       | Summary                                                        |
|------------|----------------------------------------------------------------|
| Definition | Sliding weighted average; or finite innovation model MA( $q$ ) |

| Item            | Summary                                                      |
|-----------------|--------------------------------------------------------------|
| Input/output    | Recent values/order → smooth/forecast                        |
| Steps           | Choose window → align causally → aggregate → update/validate |
| Hyperparameters | Window $w$ , weights; MA order $q$                           |
| Metrics         | Forecast MAE/RMSE, lag response, residual ACF                |
| Pros/cons       | Simple/fast / lags changes and misses structure              |
| Best use        | Baselines, monitoring, rolling ML features                   |

## Exponential Smoothing

### 1. Overview

Exponential smoothing forecasts by recursively updating latent level, trend, and seasonal components, weighting recent errors more than old ones. Simple exponential smoothing (SES) handles a stable level; Holt adds trend; Holt-Winters/ETS adds seasonality. It is a high-value forecasting baseline because it is fast, data-efficient, interpretable, and often competitive for individual business series.

### 2. Intuition

Instead of keeping a hard window, maintain a current belief. After each observation, move that belief a fraction  $\alpha$  toward the new value. A high  $\alpha$  reacts quickly; a low one remembers history. Holt and Holt-Winters similarly maintain beliefs about slope and recurring seasonal position.

### 3. Prerequisites

- Weighted averages, recursive updates, trend, seasonality.
- Loss optimization, initialization, residuals, prediction intervals.
- Additive versus multiplicative components.

### 4. Core Concepts

| Subtopic         | Meaning / why it matters                    | Example                 | Interview angle              |
|------------------|---------------------------------------------|-------------------------|------------------------------|
| Level $l_t$      | Current smoothed baseline                   | Underlying daily demand | Role of $\alpha$             |
| Trend $b_t$      | Current rate of change                      | +5 units/week           | Holt and damping             |
| Seasonal $s_t$   | Phase-specific deviation/factor             | Monday uplift           | Additive vs multiplicative   |
| Smoothing params | Update rates $\alpha, \beta, \gamma$        | High $\alpha$ adapts    | Learned vs manually set      |
| Damping $\phi$   | Shrinks trend contribution at long horizons | Growth slows            | More realistic extrapolation |

| Subtopic | Meaning / why it matters               | Example     | Interview angle                  |
|----------|----------------------------------------|-------------|----------------------------------|
| ETS      | Error-Trend-Seasonality model taxonomy | ETS(A,Ad,A) | Statistical intervals/likelihood |

## 5. Algorithm / Working Process

1. Choose SES, Holt, or Holt-Winters based on observed level/trend/seasonality.
2. Select additive or multiplicative error/seasonal form and period  $m$ .
3. Initialize level, trend, and seasonal states.
4. For each observation, compute forecast error and update states recursively.
5. Optimize smoothing parameters and initial states using likelihood/SSE.
6. Project states  $h$  steps ahead and form intervals from an ETS model or simulation.

Input is one regularly spaced series; output is point forecasts, fitted states, and optionally intervals. Training estimates a few parameters; inference recursively projects final states.

## 6. Mathematical Foundation

SES uses

$$l_t = \alpha y_t + (1-\alpha)l_{t-1}$$

$$\hat{y}_{t+h|t} = l_t, \quad 0 < \alpha < 1$$

Expanding recursion shows weights  $\alpha(1-\alpha)^k$ , hence “exponential.” Holt additive trend:

$$l_t = \alpha y_t + (1-\alpha)(l_{t-1} + b_{t-1})$$

$$b_t = \beta(l_t - l_{t-1}) + (1-\beta)b_{t-1}$$

$$\hat{y}_{t+h|t} = l_t + h b_t$$

Additive Holt-Winters:

$$l_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(l_{t-1} + b_{t-1})$$

$$b_t = \beta(l_t - l_{t-1}) + (1-\beta)b_{t-1}$$

$$s_t = \gamma(y_t - l_t) + (1-\gamma)s_{t-m}$$

$$\hat{y}_{t+h|t} = l_t + h b_t + s_{t-m+h_m}$$

where  $h_m = ((h-1) \bmod m) + 1$  selects phase. Parameters commonly minimize  $SSE = \sum e_t^2$  or maximize an explicit ETS likelihood. Damped trend replaces  $h b_t$  with  $(\phi + \dots + \phi^h)b_t$ ,  $0 < \phi < 1$ .

## 7. Practical Implementation

```
import numpy as np
from statsmodels.tsa.holtwinters import ExponentialSmoothing
```

```

rng = np.random.default_rng(1)
t = np.arange(140)
y = 40 + 0.08*t + 5*np.sin(2*np.pi*t/7) + rng.normal(0, 1, len(t))
train, test = y[:-21], y[-21:]

model = ExponentialSmoothing(
    train, trend="add", damped_trend=True,
    seasonal="add", seasonal_periods=7,
    initialization_method="estimated",
).fit(optimized=True)
pred = model.forecast(len(test))
mae = np.mean(np.abs(test - pred))
seasonal_naive = train[-7:][np.arange(len(test)) % 7]
print({"holt_winters_mae": mae,
      "seasonal_naive_mae": np.mean(np.abs(test-seasonal_naive))})

```

## 8. Code Explanation

The training cutoff leaves the final three weeks unseen. Additive states suit constant seasonal amplitude; damping prevents an indefinitely constant slope. Statsmodels jointly estimates initialization and smoothing parameters. The comparison to seasonal-naive is essential—complex smoothing has value only if it improves the appropriate baseline.

## 9. Training / Evaluation

Use rolling folds containing enough full seasonal cycles. Tune form, period, damping, and transformations on validation—not test. Inspect biased residuals and ACF. ETS models can produce model-based intervals; check empirical coverage and width. Large `alpha/gamma` can chase noise; very small values underreact to breaks. Retrain/update states as observations arrive.

## 10. Complexity and Cost

Each likelihood evaluation is  $O(n)$  for fixed seasonal period, with  $O(m)$  state and  $O(h)$  forecast cost. Optimization repeats the recursion but remains fast on CPU. Separate models for millions of sparse series may cost more operationally than a global model.

## 11. Common Use Cases

- SKU demand, inventory replenishment, and sales planning.
- Workforce, traffic, load, and call-volume forecasts.
- Fast per-series baselines and fallback models.
- Online level tracking and anomaly residuals.

## 12. Common Mistakes

- Using SES on strong trend/seasonality.
- Incorrect `seasonal_periods` or too few cycles.
- Multiplicative components with zeros/negative data.

- Assuming **alpha** is simply a learning rate with no model implication.
- Comparing only in-sample fits or skipping seasonal-naïve.
- Trusting long linear trend forecasts without damping.

## 13. Edge Cases / Limitations

Basic ETS is univariate and handles covariates poorly. Abrupt structural breaks take time to absorb; multiple/long seasonalities need extensions. Multiplicative forms require positive values. Intermittent demand and nonlinear event effects are difficult. Prediction intervals depend on the assumed error/state formulation and may be undercalibrated under regime change.

## 14. Variations

| Variation       | Change/use                                  | Importance                |
|-----------------|---------------------------------------------|---------------------------|
| SES             | Level only                                  | Core interview            |
| Holt            | Level + trend                               | Core interview            |
| Damped Holt     | Trend fades                                 | Strong practical baseline |
| Holt-Winters    | Trend + additive/multiplicative season      | Core                      |
| ETS state-space | Explicit error/trend/season likelihood      | Advanced placements       |
| Croston family  | Intermittent nonzero demand intervals/sizes | Supply-chain projects     |

## 15. Related Topics

Moving averages use a hard window; SES uses decaying infinite weights. ARIMA models autocorrelation in differenced observations/errors; some ETS and ARIMA forecasts overlap but assumptions differ. State-space form enables likelihood and intervals. Prophet is more flexible for calendars/change points; Kalman filtering generalizes recursive state updates under probabilistic dynamics.

## 16. Interview Questions

1. **Why “exponential”?** Historical weights decay geometrically as  $\alpha(1-\alpha)^k$ .
2. **What does alpha do?** Controls how rapidly level reacts to new observations.
3. **SES assumption?** Locally stable level without systematic trend/seasonality.
4. **Holt contribution?** Adds an evolving slope state.
5. **Holt-Winters contribution?** Adds recursively updated seasonal states.
6. **Additive vs multiplicative seasonality?** Constant absolute versus level-proportional amplitude.
7. **Why damping?** Constant trend extrapolation becomes implausible at long horizons.
8. **How parameters learned?** Optimize SSE or likelihood, including initial states.
9. **EMA versus SES?** Same recursion; EMA often describes smoothing, SES emphasizes forecast model.
10. **ETS versus ARIMA?** ETS models evolving components; ARIMA models differenced autocorrelation and shocks.

## 17. Practice Tasks

- Implement SES and optimize **alpha** with a validation grid.

- Compare SES/Holt/damped/Holt-Winters over rolling folds.
- Explore additive versus multiplicative demand after log transform.
- Debug a seasonal-period mismatch.
- Extend forecasts with bootstrap residual intervals and coverage checks.

## 18. Project Ideas

| Project              | Description                                  | Stack/data      | Resume value         |
|----------------------|----------------------------------------------|-----------------|----------------------|
| ETS model selector   | Backtests component forms per series         | statsmodels; M4 | Automated baselines  |
| Inventory forecaster | Damped/seasonal forecast plus reorder policy | pandas; M5      | Decision integration |
| Live KPI tracker     | Online SES states and calibrated alerts      | Python, FastAPI | Streaming inference  |

## 19. Quick Revision

- **Key idea:** recursively update level/trend/season with decaying memory.
- **Formula:**  $l_t = \alpha y_t + (1-\alpha)l_{t-1}$ .
- **Use:** interpretable univariate baseline; **metrics:** backtest MAE/MASE and coverage.
- **Trap:** wrong season form/period or undamped long trend.
- **One-liner:** "ETS is a compact state-update family; choose components by rolling validation and beat seasonal-naïve."

## 20. Final Cheat Sheet

| Item            | Summary                                                                        |
|-----------------|--------------------------------------------------------------------------------|
| Definition      | Recursive weighted forecast of latent components                               |
| Input/output    | Univariate history → states, points, intervals                                 |
| Steps           | Pick ETS form → initialize → update → optimize → project                       |
| Hyperparameters | $\alpha, \beta, \gamma, \phi, m$ , component types                             |
| Metrics         | MAE/MASE/RMSE, interval coverage                                               |
| Pros/cons       | Fast/interpretable/data-efficient / limited covariates and complex seasonality |
| Best use        | Operational baselines and stable business series                               |

## Train/Test Split for Time Series

### 1. Overview

A time-series split estimates future performance by training strictly before validation/test observations. The central rule is temporal causality: a historical simulation must use only information that would have existed at

that forecast origin. A single holdout evaluates one regime; rolling/expanding backtests evaluate many origins and reveal horizon- and regime-dependent behavior.

## 2. Intuition

To evaluate a weather forecaster for June, pretend today is each historical forecast date, hide everything after it, and issue the prediction. Randomly placing July observations in training while testing on June is like letting a student study future exam answers.

## 3. Prerequisites

- Forecast origin, horizon, lookback, gap/embargo, lags.
- Model selection versus final evaluation.
- Data leakage, preprocessing pipelines, concept drift.

## 4. Core Concepts

| Subtopic              | Meaning / why it matters                       | Example                          | Interview angle              |
|-----------------------|------------------------------------------------|----------------------------------|------------------------------|
| Holdout               | Last block reserved for final test             | Final 3 months                   | One regime limitation        |
| Expanding window      | Train start fixed, end grows                   | All history to each origin       | Uses maximum data            |
| Sliding window        | Fixed-length recent history                    | Latest 12 months                 | Handles drift                |
| Rolling origin        | Repeated simulated forecast dates              | Monthly backtests                | Robust estimates             |
| Gap/embargo           | Separation between train and evaluation        | Label matures after 7 days       | Leakage from latency/overlap |
| Nested temporal split | Inner model selection, outer unbiased estimate | Tune lags inside each outer fold | Research rigor               |

## 5. Algorithm / Working Process

1. Specify operational horizon, prediction cadence, data/label availability delays, and retraining schedule.
2. Reserve the newest interval as final test.
3. On earlier data, generate expanding or sliding validation origins.
4. At each origin, apply gap if needed; fit all preprocessing/modeling only on its training interval.
5. Predict exactly the deployment horizon without using actual intermediate outcomes unless deployment would receive them.
6. Aggregate errors by horizon, origin, entity, and decision weight; select model.
7. Refit on train+validation and evaluate once on untouched test.

Input is timestamped observations plus window rules. Output is fold indices and honest out-of-sample predictions.

## 6. Mathematical Foundation

For origin  $T_i$ , training set is commonly  $\{1, \dots, T_i\}$ , gap  $\{T_i+1, \dots, T_i+g\}$ , and validation  $\{T_i+g+1, \dots, T_i+g+h\}$ . Backtest risk is

$$R_{\text{hat}} = (1/K) \sum_{i=1}^K \left[ (1/h) \sum_{j=1}^h L(y_{T_i+g+j}, \hat{y}_{T_i+g+j|T_i}) \right]$$

Errors across folds overlap and are correlated, so naive iid standard errors are unreliable. For panel data, time boundaries normally apply to all entities. If cold-start generalization matters, add entity holdouts as a second evaluation axis.

## 7. Practical Implementation

```
import numpy as np

def rolling_splits(n, initial, horizon, step, gap=0, window=None):
    """Yield positional train/test indices for expanding or fixed windows."""
    if min(initial, horizon, step) <= 0 or gap < 0:
        raise ValueError("invalid split sizes")
    origin = initial
    while origin + gap + horizon <= n:
        start = 0 if window is None else max(0, origin - window)
        train = np.arange(start, origin)
        test = np.arange(origin + gap, origin + gap + horizon)
        yield train, test
        origin += step

for tr, te in rolling_splits(n=100, initial=50, horizon=10,
                             step=10, gap=2, window=40):
    assert tr.max() < te.min()
    print((tr[0], tr[-1]), (te[0], te[-1]))
```

## 8. Code Explanation

**origin** is the exclusive end of training. **gap** skips positions after training, modeling latency or purging overlap. **window=None** yields expanding history; an integer keeps only recent rows. The loop stops before an incomplete horizon, ensuring comparable folds. Actual pipelines must also refit scalers/encoders and enforce entity/timestamp ordering inside each split.

## 9. Training / Evaluation

Align validation with production: horizon, cadence, retraining, recursive/direct strategy, and available covariates. Use enough folds to include peaks and rare events; weight origins if recent regimes matter most. Report mean plus distribution, worst periods, per-horizon degradation, and baseline skill. Do not tune on final test. For probabilistic forecasts evaluate calibration per fold, not only point loss.

## 10. Complexity and Cost

With  $K$  folds, naive retraining multiplies model cost roughly by  $K$ . Expanding linear/statistical models may support warm starts or recursive updates, but do not change semantics for speed. Predictions require  $O(Kh)$  storage; caching immutable, point-in-time-safe features reduces repeated work. Deep-model backtests can be GPU-expensive.

## 11. Common Use Cases

- Forecast model/hyperparameter comparison.
- Churn/fraud models with temporal deployment drift.
- Financial walk-forward analysis and label embargo.
- Demand forecasts around seasons/promotions.
- Drift monitoring and champion/challenger evaluation.

## 12. Common Mistakes

- Random split or shuffled cross-validation.
- Scaling, imputing, feature selecting, or decomposing before splitting.
- Allowing overlapping target windows across train/test without a gap.
- Evaluating one-step forecasts when deployment recursively predicts 30 steps.
- Tuning repeatedly on the test block.
- Mixing revised values with the historical data vintage available then.

## 13. Edge Cases / Limitations

Sparse/short series cannot support many representative folds. One-off shocks may dominate estimates. Overlapping folds yield correlated errors. Global panel models need decisions about new entities and asynchronously starting series. Delayed labels and feature revisions require event-time snapshots. Online decisions with feedback may need replay/simulation beyond static splits.

## 14. Variations

| Variation              | Change/use                      | Importance             |
|------------------------|---------------------------------|------------------------|
| Last-block holdout     | One final contiguous future     | Minimum requirement    |
| Expanding backtest     | Accumulating history            | Standard interviews    |
| Sliding backtest       | Recent fixed window             | Drift-heavy production |
| Purged/embargo split   | Removes label overlap           | Finance/high-risk      |
| Nested backtest        | Separates tuning/evaluation     | Research               |
| Prequential evaluation | Predict then learn each arrival | Streaming              |

## 15. Related Topics

Validation design determines forecasting metric credibility. Feature stores need point-in-time joins. Concept drift motivates sliding windows and fold weighting. Direct versus recursive multi-horizon strategies must be reproduced during evaluation. Nested CV separates hyperparameter search from generalization estimates.

## 16. Interview Questions

1. **Why not random split?** It violates time order and leaks future regimes/dependent neighbors.
2. **What is rolling-origin evaluation?** Repeated train-on-past, predict-next-window simulations.
3. **Expanding versus sliding?** Expanding retains all history; sliding discards older history to adapt to drift.
4. **What is a gap?** An embargo between training and test to respect latency or remove overlap.
5. **How choose test length?** Cover deployment horizon and important cycles/regimes with enough decisions.
6. **Where fit scaler?** On each fold's training portion only.
7. **How evaluate multi-step recursion?** Generate the whole horizon exactly as production will, without true intermediate targets.
8. **Can folds overlap?** Yes, but errors correlate and uncertainty estimates must reflect it.
9. **What is final test for?** One unbiased comparison after model selection.
10. **How validate panel cold start?** Combine temporal cutoff with held-out/new-entity evaluation.

## 17. Practice Tasks

- Implement expanding, sliding, and embargo splits.
- Compare random and temporal scores on drifting synthetic data.
- Backtest 1-, 7-, and 28-step horizons.
- Find leakage from a globally fitted scaler and centered smoother.
- Extend splits for panel entities and delayed labels.

## 18. Project Ideas

| Project                    | Description                                  | Stack/data                        | Resume value                |
|----------------------------|----------------------------------------------|-----------------------------------|-----------------------------|
| Backtest toolkit           | Configurable folds, baselines, horizon plots | pandas, sklearn                   | Reusable ML evaluation      |
| Point-in-time feature demo | Replays data revisions/availability          | DuckDB, pandas                    | Production leakage literacy |
| Drift benchmark            | Compares expanding/sliding retraining        | River/sklearn; synthetic + energy | Deployment reasoning        |

## 19. Quick Revision

- **Key idea:** simulate the future from multiple historical origins.
- **Formula:** train  $\leq T$ , gap, evaluate  $T+g+1:T+g+h$ .
- **Use:** all temporal prediction; **metrics:** distribution by fold/horizon.
- **Trap:** preprocessing or labels crossing origin.
- **One-liner:** "My split reproduces production information cutoffs, horizon, and retraining cadence."

## 20. Final Cheat Sheet

| Item       | Summary                                         |
|------------|-------------------------------------------------|
| Definition | Chronological, point-in-time-correct evaluation |

| Item            | Summary                                                                      |
|-----------------|------------------------------------------------------------------------------|
| Input/output    | Ordered rows + window rules → train/validation indices/predictions           |
| Steps           | Define deployment → reserve test → roll origins → refit pipeline → aggregate |
| Hyperparameters | Initial size, horizon, step, gap, window, fold weights                       |
| Metrics         | Per-horizon/fold/segment point and calibration metrics                       |
| Pros/cons       | Realistic / computationally costly and correlated folds                      |
| Best use        | Any model deployed on future observations                                    |

## Forecasting Metrics

### 1. Overview

Forecasting metrics convert errors into model-selection and business signals. No metric is universally best: MAE estimates median-optimal absolute cost, MSE/RMSE emphasize large misses and target conditional means, percentage/scaled errors enable some comparisons, and pinball/log scores assess predictive distributions. Good evaluation includes baselines, horizons, segments, uncertainty calibration, and decision cost.

### 2. Intuition

A one-unit error matters differently when predicting one item versus one million. A hospital may punish underforecasting beds more than overforecasting; an energy operator may heavily punish extreme misses. The metric is the mathematical version of what “bad forecast” means for the decision.

### 3. Prerequisites

- Residuals, mean/median/quantiles, expectation and variance.
- Loss functions, robust statistics, temporal backtesting.
- Prediction intervals and asymmetric business costs.

### 4. Core Concepts

| Metric   | Meaning / why it matters                  | Example                         | Interview angle                   |
|----------|-------------------------------------------|---------------------------------|-----------------------------------|
| MAE      | Mean absolute error; robust, same units   | Avg miss 12 units               | Median-optimal forecast           |
| MSE/RMSE | Squared penalty; RMSE same units          | Large misses dominate           | Mean-optimal, outlier sensitivity |
| MAPE     | Mean percentage error                     | Relative miss                   | Undefined/unstable near zero      |
| WAPE     | Sum absolute error / sum actual magnitude | Volume-weighted portfolio error | Aggregation limitations           |

| Metric         | Meaning / why it matters                    | Example                 | Interview angle             |
|----------------|---------------------------------------------|-------------------------|-----------------------------|
| sMAPE          | Symmetric denominator using actual+forecast | Bounded-ish percentage  | Still unstable at both zero |
| MASE           | MAE scaled by naive in-sample MAE           | <1 beats naive scale    | Cross-series comparison     |
| Pinball loss   | Quantile forecast score                     | P90 inventory           | Asymmetric penalty          |
| Coverage/width | Calibration/sharpness of interval           | 90% interval covers 88% | Both needed                 |

## 5. Algorithm / Working Process

1. Translate decision costs and forecast output (point, quantile, distribution).
2. Select a primary metric and guardrails; define baseline and aggregation weights before tuning.
3. Produce out-of-sample predictions from temporal folds.
4. Compute per-observation errors, then aggregate by horizon, origin, entity, phase, and business value.
5. Compare skill relative to baseline and uncertainty across folds.
6. For intervals/distributions, measure calibration and sharpness; monitor bias separately.

Inputs are actuals, predictions, optional training history/weights/quantiles. Outputs are scores and diagnostic slices.

## 6. Mathematical Foundation

For  $e_t = y_t - \hat{y}_t$ :

```

MAE = (1/n) sum |e_t|
MSE = (1/n) sum e_t^2; RMSE=sqrt(MSE)
MAPE = (100/n) sum |e_t/y_t|
WAPE = 100 sum|e_t| / sum|y_t|
sMAPE= (100/n) sum 2|e_t|/(|y_t|+|y_hat_t|)
MASE = mean|e_t| / [(1/(T-m)) sum_{t=m+1}^T |y_t-y_{t-m}|]

```

Expected squared loss is minimized by the conditional mean; absolute loss by the conditional median. Quantile/pinball loss for quantile  $\tau$  and residual  $u = y - \hat{q}_\tau$  is

$$L_\tau(u) = \tau u \text{ if } u \geq 0, \text{ else } (\tau - 1)u$$

Coverage for interval  $[l_t, u_t]$  is  $\text{mean}(1\{l_t \leq y_t \leq u_t\})$ . Proper scores such as CRPS reward both calibrated and sharp distributions. Forecast skill can be  $1 - \text{metric\_model}/\text{metric\_baseline}$  when lower is better.

## 7. Practical Implementation

```

import numpy as np

def forecast_metrics(y, pred, train=None, season=1, eps=1e-12):
    y, pred = np.asarray(y, float), np.asarray(pred, float)
    err = y - pred
    out = {
        "bias": float(np.mean(err)),
        "mae": float(np.mean(np.abs(err))),
        "rmse": float(np.sqrt(np.mean(err**2))),
        "wape": float(np.sum(np.abs(err)) / max(np.sum(np.abs(y)), eps)),
    }
    if train is not None:
        train = np.asarray(train, float)
        scale = np.mean(np.abs(train[season:] - train[:-season]))
        out["mase"] = float(out["mae"] / scale) if scale > eps else np.nan
    return out

def pinball(y, q, tau):
    u = np.asarray(y) - np.asarray(q)
    return float(np.mean(np.maximum(tau*u, (tau-1)*u)))

def interval_diagnostics(y, lower, upper):
    y, lower, upper = map(np.asarray, (y, lower, upper))
    return {"coverage": np.mean((lower <= y) & (y <= upper)),
            "mean_width": np.mean(upper-lower)}

```

## 8. Code Explanation

Bias keeps the error sign (**positive** means underforecasting under this convention). WAPE uses totals and an epsilon guard, but all-zero actuals remain conceptually undefined. MASE scaling uses only training history and seasonal-naive differences. Pinball implements asymmetric quantile penalties. Coverage without width can be gamed by infinitely wide intervals, so both are returned.

## 9. Training / Evaluation

Optimize a loss aligned with evaluation when possible, but report secondary diagnostics: MAE + RMSE + bias; MASE for scale comparison; pinball across quantiles; interval coverage/width. Aggregate micro (all observations), macro (per series), and value-weighted views when relevant. Compute MASE scale independently per training fold/series. Use confidence intervals over forecast origins or blocks rather than iid rows.

## 10. Complexity and Cost

Most metrics require  $O(n)$  time and  $O(1)$  streaming accumulators. Quantile/distribution metrics cost  $O(nQ)$  for  $Q$  quantiles; CRPS from samples may be more expensive. The main cost is storing granular predictions for audit and slices, not arithmetic.

## 11. Common Use Cases

- Model ranking and hyperparameter tuning.
- SLA monitoring by forecast horizon.
- Inventory under/overstock cost optimization.
- Probabilistic capacity and risk decisions.
- Comparing accuracy across product scales and regions.

## 12. Common Mistakes

- MAPE with zeros/near zeros or negative targets.
- RMSE chosen despite a linear business cost, or MAE despite catastrophic tail cost.
- Computing scaled denominators from test data.
- Averaging percentages so tiny-volume series dominate.
- Hiding bias and segment/horizon failures behind one score.
- Evaluating intervals only by coverage or quantiles that cross.

## 13. Edge Cases / Limitations

All-zero/constant training series make MASE scale zero. Percentage metrics can be nonsensical for temperatures, returns, and signed values. Aggregation can reverse model ranking (Simpson's paradox). Delayed/revised actuals change scores. Intermittent demand needs decision-based measures and careful zero handling. Metrics do not capture downstream nonlinear constraints unless evaluated through the decision.

## 14. Variations

| Variation          | Change/use                                          | Importance               |
|--------------------|-----------------------------------------------------|--------------------------|
| RMSLE              | Squared log error; relative positive-scale emphasis | Projects                 |
| MdAE               | Median absolute error; very robust                  | Practical                |
| MASE/RMSSE         | Scale by naive absolute/squared error               | Competitions/interviews  |
| Pinball/WQL        | Quantile or weighted quantile loss                  | Probabilistic production |
| CRPS/log score     | Full predictive distribution                        | Research                 |
| Cost-weighted loss | Encodes under/over/business penalty                 | AI engineering           |

## 15. Related Topics

Regression losses define statistical targets (mean/median/quantile). Temporal validation determines unbiased metric inputs. Calibration connects predicted probability to frequency. Hierarchical forecasting introduces aggregation-level metrics. Causal decisions require policy value rather than ordinary forecast error alone.

## 16. Interview Questions

1. **MAE versus RMSE?** MAE is linear/robust and median-oriented; RMSE emphasizes extremes and mean-oriented squared loss.
2. **Why is MAPE dangerous?** Division by zero/near-zero and asymmetric behavior.
3. **What does MASE < 1 mean?** Average error is below the chosen in-sample naive scale.

4. **Why compute MASE scale on train?** Test-derived scaling leaks future information and changes the benchmark.
5. **What is bias?** Signed mean error indicating systematic under- or overforecasting.
6. **How evaluate P90?** Pinball loss at .9, empirical 90% exceedance behavior, and business cost.
7. **Coverage enough?** No; trivially wide intervals cover everything, so measure sharpness/width too.
8. **Micro versus macro average?** Micro weights observations/volume; macro gives each series equal influence.
9. **Metric for cross-series comparison?** MASE/RMSSE, with valid nonzero scales.
10. **Best metric?** The one matching decision cost, supported by diagnostics and baselines.

## 17. Practice Tasks

- Implement all formulas and test zero/negative/constant cases.
- Rank models under MAE, RMSE, MAPE, and MASE and explain reversals.
- Optimize quantiles for asymmetric inventory cost.
- Debug MASE calculated using test observations.
- Add horizon plots, block-bootstrap uncertainty, and interval calibration.

## 18. Project Ideas

| Project                  | Description                                  | Stack/data         | Resume value                |
|--------------------------|----------------------------------------------|--------------------|-----------------------------|
| Forecast scorecard       | Interactive metric/slice comparison          | pandas, Plotly; M4 | Evaluation literacy         |
| Inventory loss simulator | Maps quantiles to stockout/holding cost      | NumPy; M5          | Decision alignment          |
| Calibration monitor      | Tracks rolling quantile/interval reliability | FastAPI, DuckDB    | Production probabilistic ML |

## 19. Quick Revision

- **Key idea:** metric defines which mistakes matter and which functional is predicted.
- **Formula:**  $MASE = MAE_{model} / MAE_{naive, train}$ .
- **Use:** select/monitor forecasts; **metrics:** point + bias + calibration + business value.
- **Trap:** MAPE near zero and one aggregate score.
- **One-liner:** "I choose loss from decision cost, compare against a naive baseline, and slice by horizon and regime."

## 20. Final Cheat Sheet

| Item            | Summary                                                                 |
|-----------------|-------------------------------------------------------------------------|
| Definition      | Functions scoring point or probabilistic forecast errors                |
| Input/output    | Actuals + forecasts (+ train scale/weights) → scores                    |
| Steps           | Define cost → pick primary/guards → backtest → slice → compare baseline |
| Hyperparameters | Weights, season scale, quantiles, interval level, aggregation           |

| Item      | Summary                                              |
|-----------|------------------------------------------------------|
| Metrics   | MAE, RMSE, MASE, bias, pinball, coverage/width       |
| Pros/cons | Objective comparison / every metric embeds tradeoffs |
| Best use  | Selection, monitoring, and decision optimization     |

## ARIMA

### 1. Overview

ARIMA—AutoRegressive Integrated Moving Average—models a univariate series by differencing it to approximate stationarity, then explaining the differenced value using its past values and past innovations. It is written  $ARIMA(p, d, q)$ : AR order  $p$ , ordinary difference order  $d$ , and MA innovation order  $q$ . ARIMA is interpretable, statistically grounded, data-efficient, and an essential interview baseline for nonseasonal autocorrelated series.

### 2. Intuition

Suppose sales follow a drifting path. First model changes rather than the absolute level ( $d=1$ ). Today's change may resemble recent changes (AR terms) and may contain after-effects of recent surprises (MA terms). Forecasted changes are accumulated back to the original sales scale.

### 3. Prerequisites

- Stationarity, differencing, ACF/PACF, white noise.
- Linear regression, likelihood, residual diagnostics.
- Time-series splits, forecast intervals, lag operators.

### 4. Core Concepts

| Subtopic        | Meaning / why it matters                  | Example                        | Interview angle                          |
|-----------------|-------------------------------------------|--------------------------------|------------------------------------------|
| $AR(p)$         | Current value depends on $p$ prior values | Momentum/mean reversion        | Stationarity roots, PACF cutoff          |
| Integration $d$ | Number of ordinary differences            | Random walk needs $d=1$        | Under/over-differencing                  |
| $MA(q)$         | Current value depends on $q$ innovations  | Shock persists briefly         | ACF cutoff, invertibility                |
| Drift/intercept | Mean change or stationary mean            | Linear drift after integration | Constant interpretation changes with $d$ |
| Innovations     | One-step unpredictable residuals          | News shock                     | Not observed target lags                 |

| Subtopic                  | Meaning / why it matters              | Example             | Interview angle               |
|---------------------------|---------------------------------------|---------------------|-------------------------------|
| ARIMAX/dynamic regression | Exogenous variables plus ARIMA errors | Price and promotion | Future covariate availability |

## 5. Algorithm / Working Process

1. Plot and clean a regularly indexed series; split chronologically.
2. Stabilize variance if needed and choose minimal  $d$  using domain knowledge, tests, and plots.
3. Inspect ACF/PACF of differenced training data to propose small  $p, q$  candidates.
4. Estimate parameters via maximum likelihood/conditional sum of squares.
5. Compare AICc/BIC and rolling forecast performance.
6. Check residual mean, ACF/Ljung-Box, variance, and outliers.
7. Forecast differenced values, recursively integrate to levels, and produce intervals.

Input is a univariate history and optional known-future exogenous matrix. Training estimates coefficients and innovation variance. Inference recursively predicts conditional means, treating unknown future innovations as zero.

## 6. Mathematical Foundation

With backshift operator  $B$   $y_t = y_{t-1}$ , ARIMA is

```
phi(B)(1-B)^d y_t = c + theta(B)epsilon_t
phi(B)=1-phi_1 B-...-phi_p B^p
theta(B)=1+theta_1 B+...+theta_q B^q
epsilon_t ~ white noise(0,sigma^2)
```

An ARMA(1,1) after differencing is  $z_t = c + \phi_1 z_{t-1} + \epsilon_t + \theta_1 \epsilon_{t-1}$ . AR stationarity requires roots of  $\phi(z) = 0$  outside the unit circle; MA invertibility requires the same for  $\theta(z)$ . Gaussian likelihood is based on innovations:

$$\log L = -1/2 \sum_t [\log(2\pi\sigma_t^2) + \epsilon_t^2/\sigma_t^2]$$

AIC =  $-2\log L + 2k$ ; BIC =  $-2\log L + k \log n$ . Lower values balance in-sample likelihood and parameter count but do not replace future backtests. Forecast uncertainty grows after integration because future innovations accumulate.

## 7. Practical Implementation

```
import numpy as np
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error
```

```

rng = np.random.default_rng(9)
n = 220
e = rng.normal(0, 1, n)
y = np.zeros(n)
for t in range(1, n):
    y[t] = y[t-1] + 0.3 + 0.55*(y[t-1]-y[t-2] if t > 1 else 0) + e[t]

train, test = y[:-30], y[-30:]
fit = ARIMA(train, order=(1, 1, 0), trend="t").fit()
result = fit.get_forecast(steps=len(test))
pred = result.predicted_mean
interval = result.conf_int(alpha=0.05)
print(fit.summary())
print("MAE:", mean_absolute_error(test, pred))
print("95% interval first step:", interval[0])

```

## 8. Code Explanation

The simulated level is integrated, while its first differences follow an AR(1)-like process with drift. `order=(1,1,0)` applies one difference and one AR lag. With integration, `trend="t"` supplies a linear term in levels corresponding to drift in differences under this API. `get_forecast` returns both mean and uncertainty; model adequacy still needs residual and rolling-fold checks.

## 9. Training / Evaluation

Search a small, plausible grid and use rolling validation. Information criteria help shortlist models fit to the same training data. Evaluate MAE/MASE/RMSE and interval coverage by horizon. Residuals should approximate white noise; Ljung-Box tests joint residual autocorrelations. Large residual spikes, heteroskedasticity, or remaining seasonal ACF suggest outlier, GARCH, or SARIMA extensions. Re-estimate as regimes change.

## 10. Complexity and Cost

State-space likelihood used by modern implementations is roughly linear in `n` for fixed small orders, with a constant depending on state dimension; optimization repeats it. Memory is usually  $O(n)$  for stored results but filtering state is small. CPU is sufficient. High orders and exhaustive AutoARIMA searches increase cost and instability.

## 11. Common Use Cases

- Short-horizon demand, revenue, and operational KPI forecasts.
- Macroeconomic indicators after differencing.
- Baselines for sensor, traffic, and financial level/change series.
- Regression with autocorrelated errors and known exogenous drivers.

## 12. Common Mistakes

- Confusing MA innovations with a rolling average.
- Selecting `d` only by repeated tests and over-differencing.

- Reading sample ACF/PACF as exact order-selection rules.
- Ignoring seasonal structure or structural breaks.
- Feeding future exogenous variables not known at forecast origin.
- Reporting fitted residual accuracy instead of rolling forecasts.
- Treating a nonsignificant Ljung-Box result as proof of correct specification.

## 13. Edge Cases / Limitations

ARIMA is linear and primarily univariate; nonlinearities, many covariates, multiple seasonalities, intermittent counts, and evolving parameters challenge it. Long horizons revert toward modeled trend/mean and intervals can become wide. Missing/irregular timestamps require careful handling. Near-unit roots make parameters uncertain. Gaussian intervals can fail under heavy tails or volatility clustering.

## 14. Variations

| Variation          | What changes / when useful                     | Importance                    |
|--------------------|------------------------------------------------|-------------------------------|
| AR/MA/ARMA         | $d=0$ special cases                            | Core interview                |
| ARIMAX/SARIMAX     | Regression covariates and possibly seasonality | Projects                      |
| AutoARIMA          | Automated order search with tests/IC           | Practical, validate carefully |
| Fractional ARIMA   | Fractional $d$ for long memory                 | Research                      |
| ARIMA-GARCH        | Mean plus changing conditional variance        | Finance                       |
| Intervention model | Pulses/steps for events                        | Causal/time-series analysis   |

## 15. Related Topics

Stationarity determines differencing; ACF/PACF guide candidates; SARIMA adds seasonal polynomials; exponential smoothing models components rather than differenced correlations; state-space form performs ARIMA likelihood/forecasting; dynamic regression adds covariates. VAR generalizes autoregression to multiple endogenous series.

## 16. Interview Questions

1. **Expand ARIMA( $p, d, q$ ).** AR lag count, differencing order, innovation MA order.
2. **Why difference?** Remove unit-root/nonstationary level so ARMA relationships are stable.
3. **AR versus MA?** AR uses prior observed process values; MA uses prior unobserved innovations.
4. **ACF/PACF heuristic?** AR( $p$ ): PACF cutoff/ACF tail; MA( $q$ ): ACF cutoff/PACF tail, approximately.
5. **What is invertibility?** A stable unique innovation representation for MA parameters.
6. **What is over-differencing?** Unnecessary differencing that adds noise and negative lag-1 correlation.
7. **AIC versus BIC?** Both penalize likelihood; BIC penalizes complexity more as  $n$  grows.
8. **Why residual diagnostics?** Remaining autocorrelation means predictable structure was missed.
9. **How forecast after  $d=1$ ?** Forecast changes, cumulatively add them to the last observed level.
10. **When avoid ARIMA?** Complex nonlinear/global/panel/covariate-heavy problems or unstable regimes without extensions.

## 17. Practice Tasks

- Simulate AR, MA, and ARIMA processes and match ACF/PACF.
- Grid-search small orders with rolling MASE and AIC comparison.
- Demonstrate over-differencing a stationary AR(1).
- Diagnose autocorrelated residuals from a misspecified order.
- Add known promotion covariates and verify forecast-time availability.

## 18. Project Ideas

| Project                     | Description                                       | Stack/data                 | Resume value              |
|-----------------------------|---------------------------------------------------|----------------------------|---------------------------|
| ARIMA diagnostics workbench | Order suggestions, rolling scores, residual plots | statsmodels, Streamlit; M4 | Statistical fluency       |
| Demand ARIMAX               | Promotion/calendar regression with ARIMA errors   | pandas; retail sales       | Covariates/leakage        |
| KPI anomaly model           | ARIMA intervals flag unexpected observations      | FastAPI; server metrics    | Forecast-based monitoring |

## 19. Quick Revision

- **Key idea:** difference to stability, then model value and shock lags.
- **Formula:**  $\phi(B)(1-B)^d y_t = c + \theta(B)\epsilon_t$ .
- **Use:** small-data linear univariate autocorrelation.
- **Metrics:** rolling MASE/MAE, AIC/BIC, residual Ljung-Box, coverage.
- **Trap:** over-differencing and confusing MA with smoothing.
- **One-liner:** "ARIMA is ARMA on a minimally differenced series, validated by rolling forecasts and white-noise residuals."

## 20. Final Cheat Sheet

| Item            | Summary                                                                     |
|-----------------|-----------------------------------------------------------------------------|
| Definition      | Linear model of differenced lags and innovations                            |
| Input/output    | Univariate history (+ exogenous) → point/interval forecast                  |
| Steps           | Transform/difference → select $p, q$ → MLE → diagnose → integrate forecasts |
| Hyperparameters | $p, d, q$ , trend, exogenous terms                                          |
| Metrics         | AIC/BIC, MAE/MASE/RMSE, Ljung-Box, coverage                                 |
| Pros/cons       | Interpretable/data-efficient / linear and structure-sensitive               |
| Best use        | Short-horizon nonseasonal statistical forecasting                           |

# SARIMA

## 1. Overview

SARIMA extends ARIMA with seasonal autoregression, differencing, and moving-average innovations. It is written  $\text{ARIMA}(p, d, q) \times (P, D, Q)_m$ , where  $m$  is the seasonal period. It captures dependence at both adjacent and seasonal lags and is a classic model for monthly, quarterly, weekly, or otherwise single-seasonal series.

## 2. Intuition

Today's demand can depend on yesterday ( $p$ ) and the same weekday last week ( $P, m$ ). Its surprise can also echo from yesterday ( $q$ ) and one week ago ( $Q, m$ ). Ordinary and seasonal differencing remove changing local and seasonal levels before these relations are modeled.

## 3. Prerequisites

- All ARIMA concepts, seasonal periods, seasonal naive forecasts.
- Ordinary versus seasonal differencing, ACF/PACF.
- Calendar/covariate availability and rolling validation.

## 4. Core Concepts

| Subtopic                | Meaning / why it matters                        | Example                       | Interview angle               |
|-------------------------|-------------------------------------------------|-------------------------------|-------------------------------|
| Seasonal period $m$     | Steps in one cycle                              | 12 months/year                | Must follow sampling/domain   |
| Seasonal AR $P$         | Dependence on $y_{\{t-m\}}, y_{\{t-2m\}} \dots$ | Same month last year          | Seasonal PACF signature       |
| Seasonal difference $D$ | $(1-B^m)^D$                                     | Remove annual level pattern   | Usually 0 or 1                |
| Seasonal MA $Q$         | Dependence on innovations $m$ apart             | Holiday shock echo            | Seasonal ACF signature        |
| Multiplicative SARIMA   | Ordinary/seasonal polynomials multiply          | Interaction lag $m+1$ appears | Not multiplicative data scale |
| SARIMAX                 | Adds exogenous regressors                       | Holidays and price            | Known future values           |

## 5. Algorithm / Working Process

1. Determine sampling frequency and domain-supported seasonal period.
2. Plot seasonal subseries/ACF; choose minimal  $D$  and  $d$ .
3. Propose small ordinary and seasonal orders using lag diagnostics.
4. Fit candidates by state-space maximum likelihood.
5. Select using rolling performance plus AICc/BIC.
6. Inspect residual ACF at ordinary and seasonal lags.
7. Forecast on the differenced scale, invert both difference operators, and evaluate intervals.

Input is a regular univariate series plus optional aligned exogenous variables. Output is multi-step point/interval forecasts.

## 6. Mathematical Foundation

The multiplicative seasonal model is

$$\Phi(B)^m \phi(B) (1-B)^d (1-B^m)^D y_t = c + \Theta(B)^m \theta(B) \epsilon_t$$

For example,  $(1-\phi(B))(1-\Phi(B)^m)=1-\phi(B)-\Phi(B)^m+\phi(B)\Phi(B)^{m+1}$ , so multiplying factors creates interaction lags. Seasonal differencing is  $y_t-y_{t-m}$ . With both  $d=1, D=1$ , transformation becomes

$$(1-B)(1-B^m)y_t = y_t - y_{t-1} - y_{t-m} + y_{t-m-1}$$

Too many differences increase variance. Stationarity/invertibility apply to ordinary and seasonal polynomial roots. Likelihood, information criteria, and interval construction follow the state-space ARIMA formulation.

## 7. Practical Implementation

```
import numpy as np
from statsmodels.tsa.statespace.sarimax import SARIMAX

rng = np.random.default_rng(12)
n, m = 144, 12
t = np.arange(n)
y = 100 + .4*t + 12*np.sin(2*np.pi*t/m) + rng.normal(0, 3, n)
train, test = y[:-24], y[-24:]

fit = SARIMAX(
    train,
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, m),
    enforce_stationarity=True,
    enforce_invertibility=True,
).fit(dispatch=False)
forecast = fit.get_forecast(len(test))
pred = forecast.predicted_mean
seasonal_naive = np.resize(train[-m:], len(test))
print("SARIMA MAE:", np.mean(np.abs(test-pred)))
print("seasonal-naive MAE:", np.mean(np.abs(test-seasonal_naive)))
```

## 8. Code Explanation

The example holds out two full annual cycles. `seasonal_order=(1,1,1,12)` adds annual AR, seasonal difference, and innovation components. Root constraints prevent unstable/noninvertible estimates. The compact model is illustrative rather than automatically optimal; order choice needs small-grid rolling validation and residual diagnostics.

## 9. Training / Evaluation

Training should include several full cycles. Compare seasonal-naive and nonseasonal ARIMA. Tune small order ranges; large  $m$  and many terms cause slow/unstable fitting. Evaluate by seasonal phase and horizon. Residual ACF peaks at  $m$  indicate missed seasonal dynamics. Verify 80/95% interval coverage. Holidays may require explicit regressors rather than a regular seasonal polynomial.

## 10. Complexity and Cost

State dimension grows with seasonal order and  $m$ , making likelihood substantially heavier than ARIMA, though still usually CPU feasible. Runtime is approximately linear in observations for fixed state dimension but can scale cubically in state size within generic filtering steps. Multiple large periods are inefficient.

## 11. Common Use Cases

- Monthly sales, tourism, rainfall, and economic indicators.
- Weekly demand with annual patterns when enough history exists.
- Seasonal energy/load and call-volume forecasting.
- Regression with holidays/weather and autocorrelated seasonal errors.

## 12. Common Mistakes

- Choosing  $m$  from row count rather than actual frequency/calendar.
- Confusing multiplicative seasonal polynomials with multiplicative seasonality amplitude.
- Setting  $d=D=1$  automatically and over-differencing.
- Using SARIMA for several noninteger/long seasonalities.
- Ignoring movable holidays and missing periods.
- Selecting solely by AIC without out-of-sample seasonal-naive comparison.

## 13. Edge Cases / Limitations

SARIMA assumes regular spacing and mostly linear, stable structure. It needs multiple cycles and struggles with changing phase/amplitude, complex calendars, multiple seasonalities, sparse/intermittent targets, and very long  $m$ . Parameter identification becomes weak with short samples, and structural breaks invalidate old seasonal relationships.

## 14. Variations

| Variation           | Change/use                                      | Importance               |
|---------------------|-------------------------------------------------|--------------------------|
| SARIMAX             | Exogenous regressors with seasonal ARIMA errors | Production/projects      |
| Auto-SARIMA         | Automated stepwise order search                 | Practical, must backtest |
| Fourier + ARIMA     | Compact long/multiple seasonality in regressors | Strong alternative       |
| TBATS               | Multiple/noninteger seasonality and Box-Cox     | Advanced                 |
| Seasonal ETS        | Component-state alternative                     | Core comparison          |
| Intervention SARIMA | Pulses/steps/ramps for events                   | Econometrics             |

## 15. Related Topics

SARIMA directly extends ARIMA. Holt-Winters models seasonal states rather than seasonal lag polynomials. Prophet uses Fourier/calendar terms and change points. MSTL/TBATS target multiple seasonalities. Fourier regressors with ARIMA errors can be more parsimonious for large  $m$ .

## 16. Interview Questions

1. **Decode  $(p,d,q)(P,D,Q)_m$ .** Ordinary AR/difference/MA and seasonal AR/difference/MA at period  $m$ .
2. **What does seasonal differencing do?** Subtracts the same phase from the prior cycle.
3. **How choose  $m$ ?** From sampling cadence and real process cycle, validated empirically.
4. **Seasonal AR term?** Uses values at lags  $m, 2m, \dots$  after transformations.
5. **Why “multiplicative” SARIMA?** Ordinary and seasonal lag polynomials multiply, creating interaction lags.
6. **Does that mean multiplicative amplitude?** No; that is a different additive/multiplicative decomposition concept.
7. **How much history?** Prefer several complete cycles plus enough points to estimate ordinary dynamics.
8. **Why not huge seasonal order?** Parameter/state explosion, weak identification, overfitting.
9. **When use SARIMAX?** Relevant exogenous drivers are known/forecast at the origin.
10. **Core baseline?** Seasonal naive at period  $m$ .

## 17. Practice Tasks

- Simulate and recover a SARIMA process.
- Compare  $D=0/1$  and inspect seasonal residual ACF.
- Backtest Holt-Winters, SARIMA, and seasonal-naive.
- Debug monthly data mistakenly assigned  $m=30$ .
- Add Fourier/holiday regressors and evaluate long annual seasonality.

## 18. Project Ideas

| Project              | Description                                   | Stack/data                   | Resume value          |
|----------------------|-----------------------------------------------|------------------------------|-----------------------|
| Tourism forecaster   | Annual SARIMA with calibrated intervals       | statsmodels; monthly tourism | Seasonal statistics   |
| Grid demand SARIMAX  | Weather/calendar drivers plus seasonal errors | pandas; UCI electricity      | Exogenous forecasting |
| Seasonal model arena | Automated rolling comparison and diagnostics  | Streamlit; M4                | Model selection rigor |

## 19. Quick Revision

- **Key idea:** ARIMA dynamics at adjacent and seasonal lags.
- **Formula:**  $\Phi(B^m)\phi(B)(1-B)^d(1-B^m)^D y_t = \Theta(B^m)\theta(B)e_t$ .
- **Use:** one stable seasonal period; **metrics:** MASE vs seasonal-naive, residual seasonal ACF.
- **Trap:** wrong  $m$ , over-differencing, confusing two meanings of multiplicative.
- **One-liner:** “SARIMA adds seasonal roots and lag dynamics to ARIMA and must justify itself against seasonal-naive.”

## 20. Final Cheat Sheet

| Item            | Summary                                                                  |
|-----------------|--------------------------------------------------------------------------|
| Definition      | ARIMA with seasonal AR/difference/MA factors                             |
| Input/output    | Regular single-season series (+ exogenous) → points/intervals            |
| Steps           | Set $m$ → difference minimally → fit orders → diagnose → backtest        |
| Hyperparameters | $p, d, q, P, D, Q, m$ , trend/regressors                                 |
| Metrics         | AIC/BIC, MASE/MAE, residual ACF, coverage                                |
| Pros/cons       | Interpretable seasonal dependence / costly and rigid for complex seasons |
| Best use        | Monthly/quarterly/weekly stable seasonal series                          |

## Prophet

### 1. Overview

Prophet is an additive decomposable forecasting model popularized by Meta for business series. It combines a regularized piecewise trend, Fourier seasonalities, holiday/event regressors, and noise. Its strengths are usable defaults, interpretable components, missing-data tolerance, and calendar handling; it is not automatically superior to seasonal naive, ETS, ARIMA, or boosted trees.

### 2. Intuition

Model the forecast as transparent layers: a growth curve, a weekly pattern, a yearly pattern, named holiday effects, and leftover noise. Permit the growth slope to change at selected dates, but shrink those changes unless the data strongly support them.

### 3. Prerequisites

- Regression, regularization, trend/change points.
- Fourier seasonality, holidays, uncertainty intervals.
- Pandas datetimes and temporal validation.

### 4. Core Concepts

| Subtopic           | Meaning / why it matters            | Example                     | Interview angle                  |
|--------------------|-------------------------------------|-----------------------------|----------------------------------|
| Growth $g(t)$      | Linear or saturating logistic trend | User growth toward capacity | Cap/floor requirements           |
| Change points      | Candidate slope-change dates        | Product launch              | Prior scale controls flexibility |
| Seasonality $s(t)$ | Fourier bases for periods           | Weekly/yearly pattern       | Order vs overfit                 |

| Subtopic                     | Meaning / why it matters           | Example                  | Interview angle                  |
|------------------------------|------------------------------------|--------------------------|----------------------------------|
| Holidays $h(t)$              | Event indicators/windows           | Diwali week              | Need future event calendar       |
| Additive/multiplicative mode | Components add or scale with trend | Peak scales with traffic | Data-level multiplicativity      |
| Uncertainty                  | Trend/observation uncertainty      | Planning range           | Conditional on model assumptions |

## 5. Algorithm / Working Process

1. Provide dataframe columns  $ds$  (timestamp) and  $y$  (target), plus cap/floor for logistic growth.
2. Generate candidate change points and Fourier/calendar features.
3. Build trend, seasonality, holiday, and regressor components.
4. Estimate parameters with priors/regularization (MAP-style fitting in common usage).
5. Create future timestamps and supply known future regressors/events.
6. Predict  $\hat{y}$  and intervals; inspect components and backtest at multiple cutoffs.

Training learns component coefficients and trend changes. Inference deterministically constructs future calendar bases and combines them; uncertainty is obtained from fitted noise/trend assumptions.

## 6. Mathematical Foundation

The model is

$$y(t) = g(t) + s(t) + h(t) + x(t)^T \beta + \epsilon_t$$

For piecewise-linear growth with change-point indicators  $a_j(t)=1[t \geq s_j]$ :

$$g(t) = (k + a(t)^T \delta) t + (m + a(t)^T \gamma)$$

$$\gamma_j = -s_j \delta_j \quad \# \text{ preserves continuity}$$

Slope changes  $\delta_j$  receive a sparsity-promoting prior controlled by  $\text{changepoint\_prior\_scale}$ . Seasonality uses Fourier terms as in the seasonality chapter with coefficient regularization. Logistic growth is roughly  $C(t)/(1+\exp(-k(t-m)))$ , modified piecewise; capacity  $C(t)$  must be supplied into the future. Default uncertainty should be empirically calibrated rather than assumed exact.

## 7. Practical Implementation

```
import numpy as np
import pandas as pd
from prophet import Prophet

# df must contain timestamp column ds and numeric target y.
```

```

df = pd.DataFrame({
    "ds": pd.date_range("2023-01-01", periods=730, freq="D"),
})
df["y"] = (100 + 0.05*np.arange(len(df))
          + 10*df["ds"].dt.dayofweek.isin([5, 6]).astype(int))

train, test = df.iloc[:-60], df.iloc[-60:]
model = Prophet(
    weekly_seasonality=True,
    yearly_seasonality=True,
    seasonality_mode="additive",
    changepoint_prior_scale=0.05,
)
model.fit(train)
future = model.make_future_dataframe(periods=len(test), freq="D")
fcst = model.predict(future).tail(len(test))
mae = np.mean(np.abs(test["y"].to_numpy() - fcst["yhat"].to_numpy()))
print(fcst[["ds", "yhat", "yhat_lower", "yhat_upper"]].head(), mae)

```

## 8. Code Explanation

The final 60 days are a chronological holdout. Weekly/yearly bases and a regularized piecewise-linear trend are fit only on training data. `make_future_dataframe` supplies timestamps, not unknown target values. `predict` returns component columns and intervals. Real selection should use Prophet's time-series cross-validation or a custom rolling backtest and include seasonal-naive comparisons.

## 9. Training / Evaluation

Tune change-point prior/range, seasonality mode/order/prior, holidays, and history length through rolling cutoffs. Include full annual cycles for yearly seasonality. Report MAE/MASE/RMSE and interval coverage by horizon/holiday. Diagnose trend component, implausible future slope, component amplitude, and residual ACF. Treat external regressors as point-in-time features with future availability guaranteed.

## 10. Complexity and Cost

Feature construction is  $O(nK)$  for component dimension  $K$ ; fitting iterative regression/change-point parameters is typically manageable on CPU for single series but repeated per-series fitting can be expensive. Prediction is roughly  $O(hK)$ . Many change points, Fourier orders, holidays, and MCMC uncertainty increase cost.

## 11. Common Use Cases

- Daily business KPIs, sales, traffic, and capacity planning.
- Strong weekly/yearly calendars with named holidays.
- Analysts needing decomposed, explainable components.
- Robust baselines with missing observations and outliers.

## 12. Common Mistakes

- Assuming Prophet is universally state of the art.

- Leaving defaults without seasonal-naive/ETS/ARIMA backtests.
- Excessive change-point flexibility that fits noise.
- Forgetting caps/floors or future regressors for logistic/external inputs.
- Treating a holiday coefficient as causal impact.
- Trusting intervals without coverage evaluation.
- Using subdaily/yearly defaults without checking actual cadence/history.

## 13. Edge Cases / Limitations

Prophet is largely additive regression and may miss short autoregressive dynamics, nonlinear variable interactions, multiple entities, intermittent demand, and abrupt unmodeled regimes. It needs adequate cycles for long seasonality. Change points extrapolate uncertain recent slopes. Holiday effects require repeated or pooled evidence. Default intervals may not capture covariate, event, or regime uncertainty.

## 14. Variations

| Variation                  | Change/use                        | Importance              |
|----------------------------|-----------------------------------|-------------------------|
| Logistic growth            | Saturating trend with cap/floor   | Product growth projects |
| Multiplicative seasonality | Seasonal effect scales with trend | Common interview topic  |
| Custom seasonality         | User period/Fourier order         | Practical               |
| Extra regressors           | Known external drivers            | Production              |
| Conditional seasonality    | Pattern active under condition    | Advanced practical      |
| Bayesian/MCMC intervals    | Samples parameter uncertainty     | Research, higher cost   |

## 15. Related Topics

Prophet's trend resembles regularized piecewise regression; its seasonality is Fourier regression; holidays are event dummies. ETS adapts component states recursively; SARIMA captures lagged residual structure; boosted trees can use calendar/lags with interactions. Causal impact methods require counterfactual assumptions beyond Prophet holiday coefficients.

## 16. Interview Questions

1. **Prophet equation?** Trend + seasonality + holidays/regressors + noise.
2. **How handles trend changes?** Candidate change points with regularized slope adjustments.
3. **What controls trend flexibility?** `changepoint_prior_scale` and change-point placement/range.
4. **How seasonality represented?** Fourier sine/cosine bases.
5. **Additive versus multiplicative mode?** Constant component magnitude versus magnitude scaling with trend.
6. **When use logistic growth?** A meaningful known/forecastable capacity bounds growth.
7. **Does it handle missing dates?** It can fit irregularly observed timestamps, but missingness semantics still matter.
8. **Are holiday effects causal?** No; they are predictive associations unless causal identification exists.
9. **Why can it fail?** Weak lag modeling, unstable extrapolation, limited nonlinear interactions/global sharing.

10. **How validate?** Rolling cutoffs matching horizon, against appropriate naive/classical baselines.

## 17. Practice Tasks

- Fit weekly/yearly Prophet and plot component estimates.
- Tune change-point prior on stable versus broken trends.
- Compare additive/multiplicative seasonality on log-growing data.
- Debug a missing future regressor and timezone issue.
- Add regional holidays and evaluate each event across years.

## 18. Project Ideas

| Project                 | Description                             | Stack/data                      | Resume value               |
|-------------------------|-----------------------------------------|---------------------------------|----------------------------|
| Business KPI forecaster | Explainable trends, holidays, intervals | Prophet, Streamlit; web traffic | Stakeholder explainability |
| Capacity growth planner | Logistic scenarios with changepoints    | Prophet, Plotly; product data   | Scenario modeling          |
| Forecast benchmark      | Prophet vs ETS/SARIMA/LightGBM          | pandas; M4/M5 subset            | Honest model comparison    |

## 19. Quick Revision

- **Key idea:** regularized decomposable calendar model.
- **Formula:**  $y(t)=g(t)+s(t)+h(t)+x(t)'\beta+\epsilon$ .
- **Use:** business series with trend, calendars, missing points.
- **Metrics:** rolling MASE/MAE and interval coverage.
- **Trap:** default worship and causal interpretation of events.
- **One-liner:** "Prophet is interpretable piecewise trend plus Fourier/calendar regression, not a replacement for backtesting."

## 20. Final Cheat Sheet

| Item            | Summary                                                                   |
|-----------------|---------------------------------------------------------------------------|
| Definition      | Additive decomposable forecasting model                                   |
| Input/output    | $ds, y$ (+ events/regressors/cap) → components, point, interval           |
| Steps           | Configure growth/calendar → fit → create future → predict → backtest      |
| Hyperparameters | Change-point/seasonality priors, Fourier order, mode, cap                 |
| Metrics         | MAE/MASE/RMSE, baseline skill, coverage                                   |
| Pros/cons       | Usable/interpretable calendars / weak complex dynamics and global sharing |
| Best use        | Daily business KPIs with holidays and changing trend                      |

# LSTM for Time Series

## 1. Overview

Long Short-Term Memory (LSTM) is a gated recurrent neural network designed to carry, write, and erase information through a sequence. For time series it maps past target/covariate windows to future points, quantiles, distributions, classes, or anomaly scores. LSTMs can learn nonlinear dynamics and share patterns across many related series, but require more data/tuning than statistical baselines.

## 2. Intuition

An LSTM maintains a notebook (**cell state**). At each timestamp, a forget gate erases irrelevant notes, an input gate writes useful new evidence, and an output gate decides what part to reveal. For demand, it may retain promotional momentum while forgetting an isolated sensor glitch.

## 3. Prerequisites

- Tensors, neural networks, activations, backpropagation, gradient descent.
- Sequence windows, batching/padding/masks, scaling.
- Temporal validation and multi-step forecasting strategies.

## 4. Core Concepts

| Subtopic                             | Meaning / why it matters                                            | Example                     | Interview angle           |
|--------------------------------------|---------------------------------------------------------------------|-----------------------------|---------------------------|
| Hidden state <b><math>h_t</math></b> | Exposed sequence representation                                     | Recent demand context       | Shape/batch semantics     |
| Cell state <b><math>c_t</math></b>   | Persistent memory path                                              | Long seasonal/event context | Gradient flow             |
| Gates                                | Sigmoid-controlled forget/write/output                              | Ignore bad reading          | LSTM vs vanilla RNN       |
| Lookback                             | Input sequence length <b><math>L</math></b>                         | Prior 28 days               | More is not always better |
| Many-to-one/many                     | Last state $\rightarrow$ one target; sequence $\rightarrow$ horizon | Next day vs next week       | Output design             |
| Teacher forcing                      | Decoder sees true prior target during training                      | Seq2seq forecast            | Exposure bias             |

## 5. Algorithm / Working Process

1. Build point-in-time-valid windows  **$[batch, L, features]$**  and targets  **$[batch, horizon]$** .
2. Fit scalars on training data; encode static, past-observed, and known-future variables distinctly.
3. Initialize hidden/cell states and recurrently update them over input steps.
4. Map final/all hidden states through a head to point/quantile/distribution outputs.
5. Compute loss, backpropagate through time, clip gradients, update parameters.

- At inference, use a direct horizon head or roll recursively with predictions and known future features.

Output shape and decoding must exactly match deployment. A global LSTM trains shared weights across many series, often with entity embeddings.

## 6. Mathematical Foundation

For input  $x_t$ , previous hidden  $h_{t-1}$ , cell  $c_{t-1}$ :

```
f_t = sigmoid(W_f[x_t, h_{t-1}] + b_f)    # forget
i_t = sigmoid(W_i[x_t, h_{t-1}] + b_i)    # input
g_t = tanh(W_g[x_t, h_{t-1}] + b_g)      # candidate
c_t = f_t*c_{t-1} + i_t*g_t              # memory
o_t = sigmoid(W_o[x_t, h_{t-1}] + b_o)    # output
h_t = o_t*tanh(c_t)
```

The additive cell update creates a path that mitigates (not eliminates) vanishing gradients. Point models often minimize MSE/MAE; quantile heads use pinball loss. Backpropagation through time computes gradients across unrolled steps; gradient norm clipping uses  $g \leftarrow g * \min(1, c / ||g||)$ .

## 7. Practical Implementation

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

class LSTMForecaster(nn.Module):
    def __init__(self, n_features, hidden=32, horizon=7):
        super().__init__()
        self.lstm = nn.LSTM(n_features, hidden, batch_first=True)
        self.head = nn.Linear(hidden, horizon)

    def forward(self, x):
        sequence, _ = self.lstm(x)          # [B, L, hidden]
        return self.head(sequence[:, -1])  # [B, horizon]

def windows(y, lookback=28, horizon=7):
    xs, ys = [], []
    for end in range(lookback, len(y)-horizon+1):
        xs.append(y[end-lookback:end, None])
        ys.append(y[end:end+horizon])
    return torch.tensor(xs, dtype=torch.float32), torch.tensor(ys,
dtype=torch.float32)

torch.manual_seed(3)
t = torch.arange(500, dtype=torch.float32)
y = (0.02*t + 3*torch.sin(2*torch.pi*t/7) + 0.3*torch.randn(500)).numpy()
cut = 400
mean, std = y[:cut].mean(), y[:cut].std()
X, Y = windows((y-mean)/std)
```

```
# Keep only samples whose complete target lies before the cutoff.
n_train = cut - 28 - 7 + 1
loader = DataLoader(TensorDataset(X[:n_train], Y[:n_train]), batch_size=32,
shuffle=True)
model, loss_fn = LSTMForecaster(1), nn.MSELoss()
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
for _ in range(20):
    for xb, yb in loader:
        opt.zero_grad(); loss = loss_fn(model(xb), yb); loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0); opt.step()
with torch.no_grad():
    next_7 = model(X[n_train:n_train+1]).squeeze().numpy()*std + mean
print(next_7)
```

## 8. Code Explanation

`windows` creates causal input/target slices. Scaling uses only the first 400 timestamps. `batch_first=True` expects `[B,L,F]`. The direct head outputs all seven horizons at once, avoiding recursive error feedback. Mini-batches may shuffle already-created training windows; this does not leak validation data because recurrent state is reset for each independent window. Gradient clipping guards exploding gradients.

## 9. Training / Evaluation

Create folds before windows or track each target endpoint carefully. Fit scaling per training fold. Monitor validation loss by horizon and early-stop; tune lookback, hidden size, layers, dropout (effective between stacked layers), batch size, learning rate, and horizon head. Compare naive/ETS/tree baselines. Regularize, increase related-series data, and use entity/static features before merely enlarging the network. Evaluate inference with the same recursive/direct strategy.

## 10. Complexity and Cost

An LSTM layer costs roughly  $O(B \cdot L \cdot H(F+H))$  time and stores  $O(BLH)$  activations for backpropagation, where  $H$  is hidden size and  $F$  features. Recurrence prevents full time-step parallelization, so training is slower than convolutions/Transformers on parallel hardware for long sequences. CPU serves small models; GPU helps training/global high-volume inference.

## 11. Common Use Cases

- Global demand/load forecasting across many entities.
- Multivariate sensor and predictive-maintenance sequences.
- Traffic, weather, finance, and healthcare monitoring.
- Sequence classification, anomaly scoring, and remaining-life estimation.

## 12. Common Mistakes

- Scaling/windows constructed before split and crossing the cutoff.
- Assuming shuffled training windows equal shuffled raw temporal split.
- Incorrect tensor shape or taking the wrong LSTM output.
- Too little data for model capacity; no naive baseline.

- Recursive training/inference mismatch and teacher-forcing exposure bias.
- Ignoring known-future versus observed-only covariates.
- Using bidirectional LSTM for online forecasting when its encoder sees future positions.

## 13. Edge Cases / Limitations

Long sequences remain hard despite gates; recurrence is sequential. LSTMs extrapolate trend poorly without suitable transformations/features and may fail under regime shifts. Missing/irregular data need masks/time-gap features or specialized cells. Small univariate datasets favor statistical methods. Hidden states are less interpretable, and uncertainty requires quantile/distribution heads, ensembles, or Bayesian methods.

## 14. Variations

| Variation          | Change/use                         | Importance                                             |
|--------------------|------------------------------------|--------------------------------------------------------|
| Stacked LSTM       | Multiple recurrent layers          | Projects; overfit risk                                 |
| Bidirectional      | Reads both directions              | Imputation/classification, not causal forecast encoder |
| Encoder-decoder    | Separate recurrent horizon decoder | Multi-step interviews                                  |
| GRU                | Fewer gates/parameters             | Core comparison                                        |
| ConvLSTM           | Spatial convolution inside gates   | Weather/video research                                 |
| Probabilistic LSTM | Quantile/distribution output       | Production                                             |

## 15. Related Topics

Vanilla RNN suffers stronger vanishing gradients; GRU is simpler; TCN parallelizes causal convolutions and controls receptive field; Transformers use attention for long dependencies; state-space sequence models target long efficient context. Classical ARIMA is linear/data-efficient and more interpretable.

## 16. Interview Questions

1. **Why LSTM over RNN?** Gated additive memory improves long-range gradient/information flow.
2. **Role of forget gate?** Controls how much previous cell memory persists.
3. **Hidden versus cell state?** Hidden is exposed output; cell is internal persistent memory.
4. **Input shape in PyTorch batch-first?** `[batch, sequence, features]`.
5. **Many-to-one versus many-to-many?** One output from sequence versus output per step/horizon.
6. **What is teacher forcing?** Feeding true prior decoder targets during training.
7. **What is exposure bias?** At inference the decoder consumes its imperfect predictions, a state unseen during teacher-forced training.
8. **Direct versus recursive?** Predict all horizons jointly versus repeatedly predict next step.
9. **Why clip gradients?** Recurrent multiplication can produce exploding gradients.
10. **When not use LSTM?** Small/simple univariate data or long-context workloads better served by baselines/parallel models.

## 17. Practice Tasks

- Build one-step and direct seven-step LSTMs.
- Add known-future calendar features and static entity embeddings.
- Compare GRU/LSTM/seasonal-naive with equal backtests.
- Debug cutoff-crossing windows and wrong  $[L, B, F]$  shape.
- Add quantile heads and verify empirical coverage.

## 18. Project Ideas

| Project            | Description                              | Stack/data       | Resume value           |
|--------------------|------------------------------------------|------------------|------------------------|
| Global retail LSTM | Shared multi-SKU quantile forecasts      | PyTorch; M5      | Panels and uncertainty |
| Turbofan RUL       | Multisensor remaining-life prediction    | PyTorch; C-MAPSS | Sequence regression    |
| Traffic forecaster | Multi-horizon flow with calendar/weather | PyTorch; METR-LA | Multivariate DL system |

## 19. Quick Revision

- **Key idea:** gated recurrent memory for nonlinear sequences.
- **Formula:**  $c_t = f_t * c_{t-1} + i_t * g_t$ .
- **Use:** enough related sequential data and nonlinear dynamics.
- **Metrics:** horizon MAE/MASE, pinball/coverage.
- **Trap:** leakage in windows and recursive exposure bias.
- **One-liner:** "LSTM gates protect a persistent cell path, but I use it only when global nonlinear signal beats strong temporal baselines."

## 20. Final Cheat Sheet

| Item            | Summary                                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Definition      | Gated recurrent sequence network                                                                                  |
| Input/output    | $[B, L, F] \rightarrow$ point/quantile/distribution horizons                                                      |
| Steps           | Window/scale $\rightarrow$ recurrent gates $\rightarrow$ head $\rightarrow$ loss/BPTT $\rightarrow$ causal decode |
| Hyperparameters | Lookback, hidden, layers, dropout, LR, horizon                                                                    |
| Metrics         | MAE/MASE/RMSE, pinball, coverage                                                                                  |
| Pros/cons       | Nonlinear/global sharing / sequential, data-hungry, opaque                                                        |
| Best use        | Multivariate/panel sequences of moderate length                                                                   |

## Temporal CNN

## 1. Overview

A Temporal Convolutional Network (TCN) uses one-dimensional causal convolutions, usually with dilation and residual blocks, to model sequences. “Causal” means output at time  $t$  depends only on positions  $\leq t$ ; dilation expands the receptive field without a huge kernel. TCNs train in parallel across timestamps, have stable gradient paths, and often compete strongly with recurrent networks for forecasting and sequence labeling.

## 2. Intuition

Slide a learned pattern detector across history. A small filter might detect a sudden rise. Stacking filters lets later layers detect motifs of motifs. Dilation makes a filter inspect today, two steps ago, four steps ago, and so on—like reading a long history at increasingly coarse spacing.

## 3. Prerequisites

- 1D convolution, kernel, stride, padding, channels.
- Neural training, residual connections, dropout.
- Causal windows, multi-step loss, receptive field.

## 4. Core Concepts

| Subtopic           | Meaning / why it matters                 | Example                   | Interview angle                   |
|--------------------|------------------------------------------|---------------------------|-----------------------------------|
| Causal convolution | No future position enters current output | Left padding only         | How prevent leakage?              |
| Dilation           | Kernel taps spaced by $d$                | Lags 0,2,4 for $d=2$      | Receptive-field growth            |
| Residual block     | Input shortcut around conv stack         | Stable deep training      | Channel projection                |
| Receptive field    | History accessible to an output          | 63 prior steps            | Compute it before lookback choice |
| Channels           | Learned temporal feature maps            | Event/momentum detectors  | Tensor layout $[B, C, L]$         |
| Chomp/crop         | Remove right-side padded positions       | Preserve length/causality | Padding implementation bugs       |

## 5. Algorithm / Working Process

1. Form scaled causal windows and arrange tensors  $[batch, channels, time]$ .
2. Apply left-padded convolutions with nonlinearities, normalization/dropout, and residual paths.
3. Increase dilation by layer (often  $1, 2, 4, \dots$ ) until receptive field covers relevant history.
4. Use final timestep or a causal sequence head to predict one/multiple horizons.
5. Optimize MSE, MAE, pinball, or likelihood through standard backpropagation.
6. At inference, process the last window; cache convolution states for streaming if latency demands it.

## 6. Mathematical Foundation

A causal dilated convolution is

$$z_t = \sum_{i=0}^{k-1} w_i x_{t-d*i}$$

For layers with kernel  $k_l$ , dilation  $d_l$ , stride one, receptive field is

$$R = 1 + \sum_l (k_l - 1) d_l$$

If each residual block contains two convolutions, count both. With  $k=3$  and dilation  $[1, 2, 4, 8]$ , one conv/layer gives  $R=1+2(1+2+4+8)=31$ . Residual update  $h_{l+1}=h_l+F_l(h_l)$  creates short gradient paths. Causal padding on the left is  $(k-1)d$ ; symmetric padding without cropping leaks future positions for sequence-aligned outputs.

## 7. Practical Implementation

```
import torch
from torch import nn

class CausalConv1d(nn.Conv1d):
    def __init__(self, in_ch, out_ch, kernel_size, dilation=1):
        self.left_padding = (kernel_size - 1) * dilation
        super().__init__(in_ch, out_ch, kernel_size,
                         padding=self.left_padding, dilation=dilation)

    def forward(self, x):
        y = super().forward(x)
        return y[..., :-self.left_padding] if self.left_padding else y

class TCNForecaster(nn.Module):
    def __init__(self, n_features, channels=32, horizon=7):
        super().__init__()
        layers, in_ch = [], n_features
        for dilation in (1, 2, 4, 8):
            layers += [CausalConv1d(in_ch, channels, 3, dilation), nn.ReLU()]
            in_ch = channels
        self.net = nn.Sequential(*layers)
        self.head = nn.Linear(channels, horizon)

    def forward(self, x):
        # x: [B, L, F]
        h = self.net(x.transpose(1, 2)) # Conv1d: [B, F, L]
        return self.head(h[..., -1])

model = TCNForecaster(n_features=5, horizon=12)
assert model(torch.randn(8, 64, 5)).shape == (8, 12)
```

## 8. Code Explanation

PyTorch `Conv1d` expects channels before time. Each layer left-pads by its effective kernel width; the crop removes extra right-aligned outputs and preserves sequence length. Dilations give a 31-step receptive field in this one-convolution-per-layer example. The final causal representation feeds a direct 12-horizon head. Production TCNs commonly add residual blocks, dropout, and weight/layer normalization.

## 9. Training / Evaluation

Split before windowing and scale on training data. Ensure lookback is at least the computed receptive field; context beyond it is unused. Tune kernel, dilation schedule, block count, channels, dropout, and learning rate. Compare LSTM and naive/tree baselines under identical folds. Inspect horizon error and ablate future-known features. For classification, masks must exclude padded positions from loss.

## 10. Complexity and Cost

Dense convolution costs about  $O(B \cdot L \cdot k \cdot C_{in} \cdot C_{out})$  per layer and parallelizes over  $L$ ; dilation expands coverage without increasing parameters. Backprop memory is  $O(BLC)$  per layer. TCNs often train faster than RNNs on GPU; streaming can cache only required past activations rather than recompute the full window.

## 11. Common Use Cases

- Multi-horizon demand, load, traffic, and weather forecasting.
- Sensor anomaly detection and sequence classification.
- Audio/waveform, event, and physiological signal modeling.
- Learned temporal feature extractor combined with static/categorical inputs.

## 12. Common Mistakes

- Symmetric convolution leaking future values.
- Forgetting  $[B, C, L]$  layout.
- Claiming long context without calculating receptive field.
- Dilation gridding that misses important local interactions.
- No residual/normalization strategy in a deep network.
- Building windows or normalization across the test cutoff.

## 13. Edge Cases / Limitations

Fixed receptive fields cannot use earlier history. Very large dilation can sample sparse positions and cause gridding; mixed kernels/stacked convs help. Long seasonal periods demand depth or larger kernels. Irregular timestamps need gap features/resampling. Convolution is translation-equivariant and may need explicit absolute/calendar position. Large channel stacks consume activation memory.

## 14. Variations

| Variation           | Change/use                            | Importance             |
|---------------------|---------------------------------------|------------------------|
| Residual TCN        | Two convs + skip per block            | Standard project model |
| Depthwise separable | Cheaper channelwise + pointwise conv  | Edge deployment        |
| WaveNet             | Gated dilated causal generative convs | Research/history       |

| Variation              | Change/use                           | Importance               |
|------------------------|--------------------------------------|--------------------------|
| InceptionTime          | Multiple kernel sizes in parallel    | Classification           |
| Conv1D encoder-decoder | Down/up-sampling multiscale sequence | Segmentation/forecasting |
| TCN-attention hybrid   | Local conv + global attention        | Advanced projects        |

## 15. Related Topics

An FIR/moving-average filter is a fixed convolution; TCN learns filters. LSTM recurrence has unbounded theoretical memory but sequential execution; TCN has bounded explicit receptive field and parallel training. Transformers capture content-dependent global interactions; SSMs provide long convolutional/recurrent kernels efficiently.

## 16. Interview Questions

1. **What makes a convolution causal?** Output at  $t$  uses no input after  $t$ .
2. **What is dilation?** Spacing between kernel taps.
3. **Why dilation?** Exponential receptive-field growth with few weights/layers.
4. **Compute receptive field?**  $1 + \sum_{l=1}^L (k_l - 1)d_l$ , counting every convolution at stride one.
5. **TCN versus LSTM?** Parallel fixed-receptive convolution versus sequential gated recurrence.
6. **Why residual blocks?** Easier optimization and deep information flow.
7. **How does padding leak?** Symmetric padding plus aligned outputs can include later positions.
8. **What is gridding?** Sparse dilated taps repeatedly skip intermediate temporal relationships.
9. **Tensor shape?** Commonly input  $[B, L, F]$ , transpose to  $[B, F, L]$  for `Conv1d`.
10. **When prefer TCN?** Local/multiscale patterns, fixed context, and throughput matter.

## 17. Practice Tasks

- Compute receptive fields for two-conv residual blocks.
- Implement residual TCN and compare with LSTM.
- Write a test proving future perturbations cannot change past outputs.
- Debug symmetric-padding leakage.
- Add quantile heads and cached streaming inference.

## 18. Project Ideas

| Project               | Description                         | Stack/data                | Resume value          |
|-----------------------|-------------------------------------|---------------------------|-----------------------|
| TCN load forecast     | Direct probabilistic daily horizon  | PyTorch; UCI electricity  | Efficient DL forecast |
| ECG classifier        | Dilated multiscale rhythm detection | PyTorch; MIT-BIH          | Signal modeling       |
| Streaming anomaly TCN | Causal residual/anomaly API         | TorchScript, FastAPI; NAB | Latency and serving   |

## 19. Quick Revision

- **Key idea:** parallel causal dilated learned filters.
- **Formula:**  $z_t = \sum_i w_i x_{t-di}$ ;  $R = 1 + \sum(k_l - 1)d_l$ .
- **Use:** local/multiscale sequences; **metrics:** horizon task metric and latency.
- **Trap:** causal padding and overstated receptive field.
- **One-liner:** "TCNs replace recurrence with causal dilated convolutions whose receptive field I calculate explicitly."

## 20. Final Cheat Sheet

| Item            | Summary                                                                                        |
|-----------------|------------------------------------------------------------------------------------------------|
| Definition      | Causal dilated 1D convolutional sequence model                                                 |
| Input/output    | $[B, L, F] \rightarrow$ sequence or forecast horizon                                           |
| Steps           | Causal pad $\rightarrow$ dilated conv blocks $\rightarrow$ final state/head $\rightarrow$ loss |
| Hyperparameters | Kernel, channels, dilation, blocks, dropout, lookback                                          |
| Metrics         | MAE/MASE/pinball, throughput/latency                                                           |
| Pros/cons       | Parallel/stable/explicit context / fixed field and padding pitfalls                            |
| Best use        | Forecasting, signals, classification with finite context                                       |

## Transformer for Time Series

### 1. Overview

Time-series Transformers use attention to combine information across timestamps and variables. They add temporal position, covariates, masking, and specialized encoders/decoders to the standard Transformer. Their strengths are global content-dependent interactions, parallel training, flexible multi-horizon output, and scaling across many series; vanilla attention has quadratic sequence cost and strong data requirements.

### 2. Intuition

For each forecast, attention searches history for relevant moments rather than compressing everything into one recurrent state. When predicting Monday demand, it might attend to prior Mondays, a comparable promotion, and the latest level. The similarity is learned, while positional/calendar encodings tell the model when events occurred.

### 3. Prerequisites

- Embeddings, matrix multiplication, softmax, attention.
- Encoder/decoder, residuals, LayerNorm, masking.
- Time-series covariate roles, patches/windows, probabilistic forecasts.

### 4. Core Concepts

| Subtopic               | Meaning / why it matters                    | Example                     | Interview angle                            |
|------------------------|---------------------------------------------|-----------------------------|--------------------------------------------|
| Self-attention         | Content-dependent weighted mixing           | Compare similar days        | $Q, K, V$ formula                          |
| Causal mask            | Blocks future keys                          | Autoregressive decoder      | Encoder may use all past-window positions  |
| Position/time encoding | Supplies order and calendar/gap information | Hour/weekday/age            | Attention alone is permutation-equivariant |
| Patching               | Groups adjacent steps into tokens           | 16 readings/token           | Reduces length/noise                       |
| Direct horizon decoder | Predicts all future steps/queries           | Next 96 points              | Avoid recursive errors                     |
| Variable selection     | Weights dynamic/static covariates           | Temporal Fusion Transformer | Interpretability caveats                   |

## 5. Algorithm / Working Process

1. Separate static features, past-observed variables, and known-future variables.
2. Scale per series/group using training history; window or patch past context.
3. Project values/covariates to  $d_{model}$  and add time/position representations.
4. Apply masked/unmasked attention as allowed: the past encoder can mix within known history; an autoregressive decoder needs a causal mask.
5. Combine encoded history with future query/covariate tokens.
6. Output points, quantiles, or distribution parameters for all horizons; train end-to-end and backtest.

At inference, a direct model evaluates once; an autoregressive model iteratively appends predictions, often with key/value caching.

## 6. Mathematical Foundation

For token matrix  $X$ ,

$$Q = XW_Q, K = XW_K, V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k} + M)V$$

$M_{ij} = -\infty$  for forbidden future connections and zero otherwise. Multi-head attention performs several projections and concatenates results:

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O$$

Scaling by  $\sqrt{d_k}$  prevents dot-product variance from saturating softmax. A Transformer block combines residual attention and positionwise feed-forward layers with LayerNorm. Standard self-attention time/memory

is  $O(L^2d)/O(L^2)$  for length  $L$ ; patch size  $P$  reduces tokens to about  $L/P$ , making attention matrix about  $P^2$  times smaller.

## 7. Practical Implementation

```
import math
import torch
from torch import nn

class TimeSeriesTransformer(nn.Module):
    def __init__(self, n_features, horizon, d_model=64, heads=4, layers=2):
        super().__init__()
        self.input = nn.Linear(n_features, d_model)
        block = nn.TransformerEncoderLayer(
            d_model, heads, dim_feedforward=4*d_model,
            dropout=.1, batch_first=True, norm_first=True)
        self.encoder = nn.TransformerEncoder(block, layers)
        self.head = nn.Linear(d_model, horizon)
        self.d_model = d_model

    def forward(self, x):
        length = x.size(1)
        pos = torch.arange(length, device=x.device, dtype=x.dtype)[: , None]
        freq = torch.exp(torch.arange(0, self.d_model, 2, device=x.device,
                                     dtype=x.dtype)*(-
math.log(10000)/self.d_model))
        pe = torch.zeros(length, self.d_model, device=x.device, dtype=x.dtype)
        pe[:, 0::2], pe[:, 1::2] = torch.sin(pos*freq), torch.cos(pos*freq)
        h = self.encoder(self.input(x) + pe) # all tokens are historical
        return self.head(h[:, -1])

model = TimeSeriesTransformer(n_features=6, horizon=24)
assert model(torch.randn(16, 96, 6)).shape == (16, 24)
```

## 8. Code Explanation

Numeric features are projected into token embeddings and receive deterministic sinusoidal positions. The encoder sees the complete historical window, so no causal mask is necessary for a final forecast after that window; none of its tokens is future relative to the origin. A direct head returns 24 steps. Sequence-to-sequence training with aligned outputs would require careful causal/padding masks and known-future query tokens.

## 9. Training / Evaluation

Use large diverse panels when possible, per-series/group scaling, masks for missing/padding, and early stopping. Tune context, patch size, `d_model`, heads, layers, feed-forward width, dropout, LR/warmup, and output loss. Compare TCN/LSTM/tree/statistical baselines under identical rolling folds. Track accuracy by horizon and series scale plus GPU memory/latency. Quantile heads need crossing checks/calibration.

## 10. Complexity and Cost

Vanilla attention is  $O(BL^2d + BLd^2)$  time and  $O(BHL^2)$  attention memory. Long contexts rapidly dominate. Patching, sparse/local/linear attention, downsampling, or state-space models reduce cost. Training usually benefits from GPU mixed precision; inference cost depends heavily on direct vs autoregressive decoding and KV caching.

## 11. Common Use Cases

- Large-panel demand and probabilistic multi-horizon forecasts.
- Long-context energy, traffic, weather, and sensor modeling.
- Multivariate forecasting with static and known-future variables.
- Foundation/pretrained time-series models and zero/few-shot transfer.

## 12. Common Mistakes

- Missing causal/padding masks in aligned or autoregressive tasks.
- Treating attention weights as definitive causal explanations.
- No positional/calendar representation.
- Mixing observed-only future covariates with known-future inputs.
- Applying a huge model to one short series without baselines.
- Ignoring quadratic memory and deployment latency.
- Random window splits allowing overlapping future targets.

## 13. Edge Cases / Limitations

Transformers can be data-hungry, unstable on small datasets, and insensitive to absolute scale without normalization. Vanilla attention is expensive for high-frequency long history. Regime shifts and unseen variable distributions remain difficult. Missing/irregular time needs explicit masks/deltas. Attention may learn spurious shortcut correlations and extrapolate trends poorly.

## 14. Variations

| Variation                   | Change/use                                            | Importance               |
|-----------------------------|-------------------------------------------------------|--------------------------|
| Temporal Fusion Transformer | Gating, variable selection, static context, quantiles | Production interviews    |
| Informer/Autoformer         | Sparse attention/decomposition for long sequences     | Research awareness       |
| PatchTST                    | Patch tokens and channel-oriented processing          | Current project/research |
| TimesNet                    | 2D variation modeling from detected periods           | Research                 |
| Encoder-only direct         | History representation → whole horizon                | Practical simple design  |
| Foundation model            | Pretraining across many datasets                      | Emerging interview topic |

## 15. Related Topics

LSTMs compress sequentially; TCNs use fixed learned filters; Transformers perform content-dependent global mixing. PatchTST connects to vision patching. Temporal Fusion Transformer combines recurrence/attention/gating. Modern structured state-space models offer near-linear long-context alternatives. Positional/time embeddings connect to seasonality and calendar features.

## 16. Interview Questions

1. **Why position encoding?** Self-attention alone does not know token order.
2. **Attention formula?**  $\text{softmax}(\text{QK}^T/\sqrt{d_k}+\text{mask})V$ .
3. **Why scale dot products?** Prevent large logits and saturated, tiny-gradient softmax.
4. **When causal mask required?** Whenever an output token must not attend to later target/input positions.
5. **Why might a past-only encoder omit it?** Every position is already known at the forecast origin and only a final future head is scored.
6. **Complexity?** Quadratic in token length for vanilla attention.
7. **Why patching?** Shorter token sequence, multistep local representation, lower noise/cost.
8. **Direct versus autoregressive decoder?** One-shot correlated horizon versus iterative flexible generation/error accumulation.
9. **Can attention be causal explanation?** No; it is an internal predictive weighting without identification.
10. **When choose Transformer?** Large diverse data, long/content-dependent relations, and sufficient compute.

## 17. Practice Tasks

- Implement causal and padding masks and test them with future perturbations.
- Compare direct Transformer, LSTM, TCN under equal parameter budget.
- Patch a 1,024-step signal and measure memory/accuracy.
- Debug future-covariate leakage.
- Add quantile outputs, variable embeddings, and calibration plots.

## 18. Project Ideas

| Project                  | Description                             | Stack/data                       | Resume value           |
|--------------------------|-----------------------------------------|----------------------------------|------------------------|
| Patch demand Transformer | Global probabilistic retail forecasts   | PyTorch; M5                      | Modern architecture    |
| Long sensor model        | Sparse/patch attention anomaly forecast | PyTorch; UCR/NAB                 | Efficiency experiments |
| TFT-style planner        | Static/dynamic/known-future fusion      | PyTorch Forecasting; electricity | Production covariates  |

## 19. Quick Revision

- **Key idea:** content-dependent global temporal interaction.
- **Formula:**  $\text{softmax}(\text{QK}^T/\sqrt{d_k}+M)V$ .

- **Use:** large panel/long nonlinear context; **metrics:** horizon error, calibration, latency/memory.
- **Trap:** masks, covariate availability, quadratic cost.
- **One-liner:** "A time-series Transformer adds temporal encoding and information-safe masks to attention, trading scale and flexibility for data/compute."

## 20. Final Cheat Sheet

| Item            | Summary                                                                |
|-----------------|------------------------------------------------------------------------|
| Definition      | Attention-based temporal sequence model                                |
| Input/output    | Past/static/known-future tokens → horizon distribution/values          |
| Steps           | Scale/tokenize → position/time encode → attention blocks → decode/head |
| Hyperparameters | Context/patch, <code>d_model</code> , heads, layers, dropout, LR       |
| Metrics         | MAE/MASE/pinball/coverage plus memory/latency                          |
| Pros/cons       | Global/parallel/scalable / quadratic and data-hungry                   |
| Best use        | Large multivariate panels and long content-dependent histories         |

## State Space Models

### 1. Overview

A state space model (SSM) represents observed data as noisy measurements of an unobserved state that evolves over time. Classical linear Gaussian SSMs unify local trends, seasonal models, regression, ARIMA, and Kalman filtering. Nonlinear/non-Gaussian SSMs use extended/unscented Kalman or particle methods. Modern structured neural SSMs reinterpret long sequence modeling as efficient recurrence/convolution.

### 2. Intuition

A radar sees noisy positions, not the true position and velocity. The hidden state contains both; a transition model predicts how they evolve, and an observation model explains sensor readings. Each new reading updates belief about the state, which then produces a future distribution.

### 3. Prerequisites

- Linear algebra, multivariate Gaussian distributions, conditional probability.
- Markov property, likelihood, latent variables.
- Trend/seasonality and Kalman filter basics; for neural SSMs, discretization/convolution.

### 4. Core Concepts

| Subtopic                      | Meaning / why it matters       | Example          | Interview angle                |
|-------------------------------|--------------------------------|------------------|--------------------------------|
| Latent state <code>z_t</code> | Compact hidden process summary | Level + velocity | State is not directly observed |

| Subtopic              | Meaning / why it matters                 | Example                 | Interview angle                             |
|-----------------------|------------------------------------------|-------------------------|---------------------------------------------|
| Transition $F$        | Dynamics from $t-1$ to $t$               | Constant velocity       | Markov assumption                           |
| Observation $H$       | Maps state to measurements               | Observe position only   | Partial observability                       |
| Process noise $Q$     | Unmodeled dynamic variation              | Acceleration            | Flexibility vs stability                    |
| Measurement noise $R$ | Sensor uncertainty                       | GPS error               | Controls trust in data                      |
| Filtering/smoothing   | Infer current state / revise past states | Online vs retrospective | Forecasting cannot use smoother future info |

## 5. Algorithm / Working Process

1. Define state components and their transition; define how observations/covariates relate to state.
2. Specify initial state distribution and process/measurement noise.
3. Predict the next state distribution from previous posterior.
4. On receiving  $y_t$ , update state belief through its likelihood.
5. Accumulate predictive likelihood to estimate unknown parameters (MLE/EM/Bayesian inference).
6. Forecast by propagating state without measurement updates; optionally smooth historical states offline.

Input is observations, controls/covariates, and model matrices/functions. Output is filtered/smoothed latent distributions, likelihood, and probabilistic forecasts.

## 6. Mathematical Foundation

Linear Gaussian SSM:

$$\begin{aligned}
 z_t &= F_t z_{t-1} + B_t u_t + w_t, & w_t &\sim N(0, Q_t) \\
 y_t &= H_t z_t + D_t x_t + v_t, & v_t &\sim N(0, R_t) \\
 z_0 &\sim N(m_0, P_0)
 \end{aligned}$$

The Markov assumptions are  $p(z_t | z_{0:t-1}) = p(z_t | z_{t-1})$  and observations conditionally independent given state. Filtering factorizes likelihood as  $p(y_{1:T}) = \prod_t p(y_t | y_{1:t-1})$ . A local linear trend uses state  $[level, slope]$  and  $F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ ,  $H = \begin{bmatrix} 1 & 0 \end{bmatrix}$ . Multi-step state mean/covariance propagate as

$$\begin{aligned}
 m_{t+h} &= F^h m_t \\
 P_{t+h} &= F^h P_t (F^h)^T + \sum_{i=0}^{h-1} F^i Q (F^i)^T
 \end{aligned}$$

so uncertainty naturally grows. Kalman filtering is exact for the linear Gaussian case.

## 7. Practical Implementation

```

import numpy as np

def simulate_local_trend(n=100, q_level=.05, q_slope=.005, r=1.0, seed=0):
    rng = np.random.default_rng(seed)
    F = np.array([[1., 1.], [0., 1.]])
    H = np.array([[1., 0.]])
    Q = np.diag([q_level, q_slope])
    state = np.array([0., .2])
    states, observations = [], []
    for _ in range(n):
        state = F @ state + rng.multivariate_normal(np.zeros(2), Q)
        y = (H @ state).item() + rng.normal(0, np.sqrt(r))
        states.append(state.copy()); observations.append(y)
    return np.asarray(states), np.asarray(observations), F, H, Q, r

states, y, F, H, Q, R = simulate_local_trend()
assert states.shape == (100, 2) and y.shape == (100,)
print("hidden level vs noisy observation:", states[-1, 0], y[-1])

```

## 8. Code Explanation

The two-dimensional state holds level and slope. **F** advances level by one slope unit while retaining slope. Small process covariance permits each state to drift; scalar **R** adds observation noise. This generator makes the modeling assumptions explicit. The Kalman chapter implements inference for the generated model.

## 9. Training / Evaluation

Estimate **Q**, **R**, regression parameters, and initial state via prediction-error likelihood, EM, or Bayesian priors. Identifiability is important: large **Q**/small **R** interprets variation as real state movement; the reverse calls it measurement noise. Use rolling forecast likelihood/error and interval coverage. Inspect standardized innovations for bias, autocorrelation, and variance. Never evaluate a causal forecast using smoothed states that saw later observations.

## 10. Complexity and Cost

Dense Kalman updates for state size **k** and observation size **m** are roughly  $O(k^3+m^3)$  per step, often much less with structure/small matrices; memory can be  $O(k^2)$  online. Particle filtering costs  $O(N_{\text{particles}})$  times transition/likelihood. Structured neural SSMs exploit diagonal/low-rank dynamics and parallel scans/convolutions for near-linear sequence scaling.

## 11. Common Use Cases

- Tracking position, velocity, sensor fusion, and navigation.
- Local level/trend/seasonal forecasting with missing observations.
- Macroeconomic latent factors and nowcasting.
- Control systems, signal processing, and online anomaly detection.
- Long-sequence neural modeling via structured/selective SSMs.

## 12. Common Mistakes

- Treating state as directly observed truth.
- Confusing filter (past/current data) with smoother (also future data).
- Arbitrarily setting  $Q, R$  without calibration.
- Wrong state definition or unobservable redundant components.
- Using linear Gaussian inference for strong nonlinear/heavy-tailed dynamics without checking.
- Conflating all classical SSMs with modern neural "SSM" architectures.

## 13. Edge Cases / Limitations

Misspecified dynamics produce confident bad states. State components may be unidentifiable/unobservable. Nonlinear and multimodal posterior distributions make Gaussian filters inaccurate; particle filters can collapse in high dimension. Abrupt regime changes need switching/robust models. Classical matrices scale poorly with huge states unless structured.

## 14. Variations

| Variation                       | Change/use                               | Importance           |
|---------------------------------|------------------------------------------|----------------------|
| Linear Gaussian                 | Exact Kalman inference                   | Core interview       |
| Dynamic linear model            | Trend/season/regression states           | Forecasting projects |
| Extended/unscented KF           | Approximate nonlinear transforms         | Robotics             |
| Particle filter                 | Samples non-Gaussian/nonlinear posterior | Research             |
| Switching SSM/HMM               | Discrete regimes alter dynamics          | Advanced             |
| Structured/selective neural SSM | Efficient learned long-sequence dynamics | Modern research      |

## 15. Related Topics

Kalman filter is the inference algorithm for linear Gaussian SSMs. Hidden Markov models use discrete states. ARIMA/ETS have state-space representations. Gaussian processes model distributions over functions rather than explicit Markov state. RNNs learn nonlinear state recurrence; structured SSMs impose efficient linear dynamics plus nonlinear mixing.

## 16. Interview Questions

1. **What is a state?** A latent sufficient summary of history for predicting the future under the model.
2. **Transition versus observation model?** How state evolves versus how state generates measured data.
3. **Process versus measurement noise?** Uncertainty in dynamics versus sensors.
4. **Filtering versus prediction?** Update current state with data versus propagate it forward.
5. **Filtering versus smoothing?** Smoothing revises past states using future observations.
6. **Why Markov?** Given current state, earlier states add no predictive information for next state.
7. **How handle missing observation?** Perform prediction and skip measurement update.
8. **What is observability?** Whether state can be inferred from sequences of outputs.
9. **ARIMA connection?** ARIMA can be represented and estimated as an SSM.

10. **Modern neural SSM appeal?** Long context with structured near-linear computation and parallelizable training.

## 17. Practice Tasks

- Design state matrices for level, trend, and seasonal components.
- Simulate different  $Q/R$  ratios and interpret trajectories.
- Compare filtering and smoothing on noisy position data.
- Debug a forecast using future-smoothed states.
- Implement nonlinear particle tracking or explore a structured SSM library.

## 18. Project Ideas

| Project                | Description                                           | Stack/data                | Resume value               |
|------------------------|-------------------------------------------------------|---------------------------|----------------------------|
| GPS tracker            | Fuses noisy position into location/velocity estimates | NumPy; synthetic GPS      | Linear algebra/probability |
| Dynamic demand model   | Latent trend/season with uncertainty                  | statsmodels/PyMC; retail  | Probabilistic forecasting  |
| SSM sequence benchmark | TCN/LSTM/structured SSM on long signals               | PyTorch; Long Range Arena | Research engineering       |

## 19. Quick Revision

- **Key idea:** noisy observations reveal an evolving hidden state.
- **Formula:**  $z_t = Fz_{t-1} + w_t$ ,  $y_t = Hz_t + v_t$ .
- **Use:** latent dynamics/online probabilistic estimation.
- **Metrics:** innovation likelihood, forecast error, interval calibration.
- **Trap:** filtering vs smoothing leakage and  $Q/R$  identifiability.
- **One-liner:** "An SSM separates system dynamics from measurement noise and forecasts by propagating a posterior over latent state."

## 20. Final Cheat Sheet

| Item            | Summary                                                                             |
|-----------------|-------------------------------------------------------------------------------------|
| Definition      | Latent Markov dynamics plus observation model                                       |
| Input/output    | Noisy sequence/controls → latent posterior and forecasts                            |
| Steps           | Define state → predict → observe/update → learn params → propagate                  |
| Hyperparameters | State size, $F, H, Q, R$ , nonlinear/robust form                                    |
| Metrics         | Predictive likelihood, MAE/RMSE, innovation diagnostics, coverage                   |
| Pros/cons       | Principled online uncertainty/missing data / specification and inference complexity |
| Best use        | Tracking, components, sensor fusion, probabilistic forecasting                      |

# Kalman Filter

## 1. Overview

The Kalman filter is an exact recursive Bayesian inference algorithm for linear dynamical systems with Gaussian noise. It alternates between predicting the hidden state and correcting that prediction with a new measurement. It provides an optimal minimum-mean-squared-error state estimate under its assumptions, along with uncertainty, and is widely used in navigation, tracking, sensor fusion, control, signal processing, and dynamic forecasting.

## 2. Intuition

Suppose GPS says a vehicle is at 102 m, while a motion model predicts 100 m. If GPS is precise and the model uncertain, move close to 102; if GPS is noisy and dynamics reliable, stay near 100. The Kalman gain computes this trust automatically from uncertainties. After correction, repeat when the next measurement arrives.

## 3. Prerequisites

- Matrix multiplication, transpose, inverse/linear solve, covariance.
- Multivariate Gaussian conditioning and Bayes' rule.
- State-space transition/observation models and Markov assumption.
- Numerical stability and temporal evaluation.

## 4. Core Concepts

| Subtopic                    | Meaning / why it matters                       | Example                    | Interview angle                    |
|-----------------------------|------------------------------------------------|----------------------------|------------------------------------|
| Prior/prediction            | State belief before seeing current measurement | Physics-predicted position | Time update equations              |
| Innovation $r_t$            | Measurement minus predicted measurement        | GPS residual               | Should be white/calibrated         |
| Innovation covariance $S_t$ | Uncertainty of innovation                      | Model + sensor uncertainty | Used in gain/likelihood            |
| Kalman gain $K_t$           | Matrix balancing prior and measurement         | Trust sensor vs model      | Limits as $R \rightarrow 0/\infty$ |
| Posterior/update            | Corrected state/covariance                     | Fused estimate             | Measurement update                 |
| Riccati recursion           | Covariance evolution                           | Steady-state gain          | Numerical/observability behavior   |

## 5. Algorithm / Working Process

1. Initialize state mean  $m_0$  and covariance  $P_0$ .
2. **Predict:** propagate mean through dynamics and add process uncertainty.
3. Predict measurement; compute innovation and its covariance.
4. **Update:** compute Kalman gain and correct state mean/covariance.

5. Accumulate innovation log-likelihood for parameter estimation/diagnostics.
6. Repeat online; for  $h$ -step forecast, repeat prediction without measurement updates.

Input is model matrices/noise plus a stream of possibly missing measurements. Output at each time is a Gaussian filtered state and predictive observation distribution. Training may estimate  $Q, R, F, H$ ; filtering itself is inference, not gradient training.

## 6. Mathematical Foundation

For  $z_t = Fz_{t-1} + Bu_t + w_t, y_t = Hz_t + v_t$ :

```
# Predict
m_t^- = F m_{t-1} + B u_t
P_t^- = F P_{t-1} F^T + Q

# Innovation
r_t = y_t - H m_t^-
S_t = H P_t^- H^T + R

# Update
K_t = P_t^- H^T S_t^-1
m_t = m_t^- + K_t r_t
P_t = (I - K_t H) P_t^-
```

For numerical stability, use a linear solve rather than explicit inverse and Joseph covariance form

$$P_t = (I - KH)P_t^- (I - KH)^T + K R K^T.$$

Gaussian conditioning yields these equations. As  $R \rightarrow 0$ , a directly observed component is trusted strongly; as  $R \rightarrow \infty, K \rightarrow 0$  and measurement is ignored. Negative log predictive likelihood per step contains  $\log |S_t| + r_t^T S_t^{-1} r_t$  plus constants.

## 7. Practical Implementation

```
import numpy as np

class KalmanFilter:
    def __init__(self, F, H, Q, R, mean, covariance):
        self.F, self.H = np.asarray(F, float), np.asarray(H, float)
        self.Q, self.R = np.asarray(Q, float), np.atleast_2d(R).astype(float)
        self.mean, self.P = np.asarray(mean, float), np.asarray(covariance, float)

    def predict(self):
        self.mean = self.F @ self.mean
        self.P = self.F @ self.P @ self.F.T + self.Q
        return self.mean.copy(), self.P.copy()
```

```

def update(self, observation):
    if observation is None or np.isnan(observation):
        return self.mean.copy(), self.P.copy() # missing: keep prediction
    y = np.atleast_1d(observation).astype(float)
    innovation = y - self.H @ self.mean
    S = self.H @ self.P @ self.H.T + self.R
    # K = P H' S^-1, computed via solve to avoid explicit inverse
    K = np.linalg.solve(S, self.H @ self.P).T
    identity = np.eye(len(self.mean))
    self.mean = self.mean + K @ innovation
    A = identity - K @ self.H
    self.P = A @ self.P @ A.T + K @ self.R @ K.T # Joseph form
    return self.mean.copy(), self.P.copy()

dt = 1.0
kf = KalmanFilter(
    F=[[1, dt], [0, 1]], H=[[1, 0]],
    Q=np.diag([0.05, 0.01]), R=[[1.0]],
    mean=[0, 0], covariance=np.eye(2)*10,
)
measurements = [0.2, 1.1, 1.9, None, 4.2]
for measurement in measurements:
    kf.predict(); estimate, uncertainty = kf.update(measurement)
print("position, velocity:", estimate, "position variance:", uncertainty[0, 0])

```

## 8. Code Explanation

The state is `[position, velocity]`; `F` implements constant velocity and `H` observes position only. Prediction increases uncertainty through `Q`. Update forms the residual, solves for the gain, and uses Joseph form to preserve symmetry/positive semidefiniteness under floating-point error. A missing observation skips correction while uncertainty continues to grow.

## 9. Training / Evaluation

Estimate noise matrices using domain calibration, maximum innovation likelihood, or EM. Tune on historical training folds and evaluate future filtered forecasts, not smoothed estimates. Standardized innovations  $S_t^{-1/2}r_t$  should be zero-mean, unit-scale, and uncorrelated. Track RMSE/MAE, negative log likelihood, normalized estimation error when ground-truth state exists, and interval coverage. Stress-test dropouts, outliers, and model mismatch.

## 10. Complexity and Cost

Dense filtering is approximately  $O(k^3 + m^3)$  per observation for state dimension `k` and measurement dimension `m`, dominated by covariance operations/solve. Small tracking states are extremely cheap and run on embedded CPUs. Online memory is  $O(k^2)$ . Square-root/information filters improve stability or exploit sparse high-dimensional systems.

## 11. Common Use Cases

- GPS/IMU/radar sensor fusion and object tracking.

- Robotics, aerospace, autonomous systems, and control.
- Dynamic trend estimation and nowcasting.
- Noise reduction, missing sensor interpolation, and online anomaly detection.
- Time-varying regression coefficients.

## 12. Common Mistakes

- Running update before prediction with mismatched time indexing.
- Treating  $Q$  and  $R$  as arbitrary “smoothing knobs” without units/calibration.
- Explicit matrix inverse instead of stable solve/factorization.
- Confusing filtered state with a future-data smoother.
- Using the simple covariance update in numerically fragile settings.
- Assuming optimality under nonlinear, non-Gaussian, or misspecified dynamics.
- Ignoring variable  $dt$  for irregular sampling.

## 13. Edge Cases / Limitations

Outliers can pull a Gaussian update severely. Nonlinear observation/dynamics require approximate EKF/UKF or particles. Multimodal ambiguity cannot be represented by one Gaussian. Poor observability causes uncertainty or unstable estimates. Incorrect  $Q/R$  makes the filter sluggish or noisy. Singular/ill-conditioned covariance needs robust factorization and careful model design.

## 14. Variations

| Variation                    | What changes / when useful                         | Importance                 |
|------------------------------|----------------------------------------------------|----------------------------|
| Extended KF                  | Linearizes nonlinear functions with Jacobians      | Robotics core              |
| Unscented KF                 | Propagates sigma points; no Jacobian               | Nonlinear tracking         |
| Ensemble KF                  | Monte Carlo covariance for large systems           | Weather research           |
| Information filter           | Tracks precision; useful sparse/distributed fusion | Advanced                   |
| Square-root KF               | Factors covariance for numerical stability         | Production safety-critical |
| Rauch–Tung–Striebel smoother | Backward pass revises historical states            | Offline analysis only      |

## 15. Related Topics

State-space models define the system; Kalman is their linear-Gaussian inference. Alpha-beta filters are simplified steady-gain trackers. Bayesian filtering generalizes predict/update; particle filters represent non-Gaussian posteriors. LSTMs learn state updates without explicit uncertainty, while neural Kalman hybrids combine physics and learned components.

## 16. Interview Questions

1. **Two Kalman phases?** Predict state/covariance, then update using measurement innovation.

2. **What is innovation?** Difference between observed and predicted measurement.
3. **What does gain do?** Balances correction using prior and measurement uncertainty.
4. **If measurement noise rises?** Gain falls; model prediction is trusted more.
5. **If process noise rises?** Predicted covariance grows; measurements usually receive more weight.
6. **Why covariance grows without observations?** Uncertain process noise accumulates under prediction.
7. **Why no explicit inverse?** Linear solves/factorizations are more stable and efficient.
8. **Filter versus smoother?** Filter uses data through  $t$ ; smoother also uses data after  $t$ .
9. **Assumptions for exact optimality?** Linear dynamics/observations, Gaussian independent noise, correct model.
10. **How handle missing measurement?** Perform prediction and omit update.

## 17. Practice Tasks

- Implement scalar Kalman equations and derive steady-state gain.
- Track 2D position/velocity with variable  $dt$ .
- Sweep  $Q/R$  and visualize lag/noise tradeoff.
- Debug a covariance losing symmetry/positive definiteness.
- Add Mahalanobis innovation gating for outliers and compare EKF/UKF.

## 18. Project Ideas

| Project              | Description                                    | Stack/data                  | Resume value            |
|----------------------|------------------------------------------------|-----------------------------|-------------------------|
| GPS-IMU fusion       | Tracks 2D motion through noisy/dropout sensors | NumPy; KITTI/synthetic      | Robotics estimation     |
| Multi-object tracker | Kalman motion + assignment                     | OpenCV, scipy; MOTChallenge | CV systems              |
| Dynamic KPI filter   | Local trend/interval/anomaly streaming API     | NumPy, FastAPI              | Online probabilistic ML |

## 19. Quick Revision

- **Key idea:** predict hidden Gaussian state, correct with uncertainty-weighted innovation.
- **Formula:**  $K = P^- H' (H P^- H' + R)^{-1}$ ;  $m = m^- + K(y - H m^-)$ .
- **Use:** linear noisy dynamic systems; **metrics:** RMSE/NLL/coverage and innovation whiteness.
- **Trap:** filter vs smoother,  $Q/R$ , explicit inverse.
- **One-liner:** "Kalman filtering is recursive Gaussian conditioning in a linear state-space model."

## 20. Final Cheat Sheet

| Item            | Summary                                                                                  |
|-----------------|------------------------------------------------------------------------------------------|
| Definition      | Exact linear-Gaussian recursive state estimator                                          |
| Input/output    | Model + measurements $\rightarrow$ posterior state mean/covariance                       |
| Steps           | Predict mean/covariance $\rightarrow$ innovation $\rightarrow$ gain $\rightarrow$ update |
| Hyperparameters | $F, H, Q, R$ , initial mean/covariance, $dt$                                             |

| Item      | Summary                                                              |
|-----------|----------------------------------------------------------------------|
| Metrics   | State/forecast RMSE, NLL, innovation ACF, interval coverage          |
| Pros/cons | Fast online uncertainty/fusion / linear-Gaussian and model-sensitive |
| Best use  | Tracking, sensors, control, dynamic components                       |

## Causal Forecasting

### 1. Overview

Causal forecasting predicts future outcomes under a specified intervention, not merely under the continuation of observed correlations. Ordinary forecasting asks  $E[Y_{t+h} | \text{history}]$ ; causal forecasting asks a counterfactual such as  $E[Y_{t+h}(\text{do}(A=a))]$ , often comparing policies: “What will sales be if price is reduced?” It supports pricing, marketing, policy, medicine, operations, and planning when actions change the future.

The term is also sometimes used loosely for causal/left-to-right models that do not look into the future. That is temporal causality, not causal inference. Interviews reward making this distinction immediately.

### 2. Intuition

Umbrella sales predict rain, but giving away umbrellas does not cause rain. A predictive model can use umbrella sales to forecast weather; a causal decision model must determine what changes if we intervene. Similarly, discounts correlate with low demand because managers target weak periods; naive regression may conclude discounts reduce sales. Causal forecasting must adjust for that treatment assignment and evolving confounding.

### 3. Prerequisites

- Potential outcomes, DAGs, confounding, selection bias, and interventions.
- Regression, propensity scores, doubly robust estimation.
- Time-varying treatments/covariates, temporal backtesting.
- Experiments, difference-in-differences, synthetic control, uncertainty.

### 4. Core Concepts

| Subtopic                        | Meaning / why it matters                  | Example                       | Interview angle                    |
|---------------------------------|-------------------------------------------|-------------------------------|------------------------------------|
| Potential outcome<br>$Y_t(a)$   | Outcome that would occur under action $a$ | Demand under 10% discount     | Fundamental missing counterfactual |
| Treatment/intervention<br>$A_t$ | Controllable action/exposure              | Price, ad, medication         | Clearly define timing/version      |
| Confounder $X_t$                | Causes both action and outcome            | Expected weak demand triggers | Adjustment and time ordering       |

| Subtopic                 | Meaning / why it matters                              | Example                                | Interview angle                        |
|--------------------------|-------------------------------------------------------|----------------------------------------|----------------------------------------|
|                          |                                                       | promotion                              |                                        |
| Time-varying confounding | Past treatment changes future confounders/treatment   | Medication affects next health score   | Standard adjustment may bias           |
| Counterfactual baseline  | Outcome without intervention                          | Post-launch sales absent campaign      | Synthetic control/BSTS                 |
| Identification           | Assumptions connect observed data to causal estimand  | Exchangeability, positivity            | Cannot solve with model accuracy alone |
| SUTVA/interference       | One unit's treatment does not alter another's outcome | Competitor/store spillovers violate it | Network/spatial effects                |

## 5. Algorithm / Working Process

1. State unit, time zero, treatment strategy, outcome horizon, target population, and estimand (ATE, ATT, policy value, cumulative effect).
2. Draw a temporal DAG; distinguish pre-treatment confounders, mediators, colliders, and post-treatment variables.
3. Choose identification design: randomized experiment, backdoor adjustment, g-formula, inverse probability weighting, synthetic control, interrupted time series, difference-in-differences, or instrumental variable.
4. Build point-in-time data and check overlap/positivity, pretrends, anticipation, spillovers, and measurement timing.
5. Estimate nuisance models/counterfactual trajectories with cross-fitting or pre-intervention training as appropriate.
6. Compute intervention forecasts/effects with uncertainty; run placebo, sensitivity, and falsification tests.
7. Validate factual predictive calibration separately from causal assumptions and, when possible, compare with experimental results.

Input is longitudinal outcomes, treatments, confounders, units, and intervention scenario. Output is potential-outcome forecasts, contrasts, cumulative impact, and uncertainty.

## 6. Mathematical Foundation

For binary treatment, individual effect is  $Y_{t(1)} - Y_{t(0)}$  but only one is observed. Average treatment effect at horizon  $h$ :

$$ATE_h = E[Y_{t+h}(1) - Y_{t+h}(0)]$$

Under consistency, positivity, and conditional exchangeability  $(Y(0), Y(1)) \perp A \mid X$ , backdoor identification gives

$$E[Y(a)] = E_X[E[Y \mid A=a, X]].$$

Inverse propensity weighting estimates  $E[A Y/e(X) - (1-A)Y/(1-e(X))]$ , where  $e(X)=P(A=1|X)$ . A doubly robust ATE estimator combines outcome models  $\mu_a(X)$  and propensity:

$$\begin{aligned} \tau_{\text{hat}} = & \text{mean}[\mu_1(X) - \mu_0(X) \\ & + A(Y - \mu_1(X))/e(X) \\ & - (1-A)(Y - \mu_0(X))/(1-e(X))]. \end{aligned}$$

For time-varying strategies, the longitudinal g-formula integrates sequential outcomes/covariates under imposed treatment history; marginal structural models use products of treatment weights. Difference-in-differences estimates  $(\text{post-pre})_{\text{treated}} - (\text{post-pre})_{\text{control}}$  under parallel trends. Synthetic control predicts treated counterfactual from a weighted donor combination.

## 7. Practical Implementation

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.model_selection import KFold, cross_val_predict

def doubly_robust_ate(df):
    """Cross-fitted AIPW for a static binary treatment; not longitudinal g-
    methods."""
    X = df[["pre_demand", "price_before", "region_score"]].to_numpy()
    a, y = df["promotion"].to_numpy(), df["future_demand"].to_numpy()
    folds = KFold(5, shuffle=True, random_state=0)
    propensity = cross_val_predict(LogisticRegression(max_iter=1000), X, a,
                                   cv=folds, method="predict_proba")[:, 1]
    mu0 = np.empty(len(df)); mu1 = np.empty(len(df))
    for train, valid in folds.split(X):
        outcome = LinearRegression().fit(np.c_[X[train], a[train]], y[train])
        mu0[valid] = outcome.predict(np.c_[X[valid], np.zeros(len(valid))])
        mu1[valid] = outcome.predict(np.c_[X[valid], np.ones(len(valid))])
    e = np.clip(propensity, .02, .98) # diagnose overlap; clipping is not a cure
    score = mu1-mu0 + a*(y-mu1)/e - (1-a)*(y-mu0)/(1-e)
    return score.mean(), score.std(ddof=1)/np.sqrt(len(score))

# Required columns must be measured before promotion assignment and outcome.
# ate, standard_error = doubly_robust_ate(data)
```

## 8. Code Explanation

The propensity estimates treatment assignment from pre-treatment confounders. Cross-fitted outcome/propensity predictions reduce own-observation overfitting. The augmented inverse-probability score is consistent if either propensity or outcome model is correctly specified under the identification assumptions.

Clipping prevents numerical explosions but signals an overlap problem; it cannot manufacture support. This static example is inappropriate when treatment-confounder feedback requires longitudinal g-methods.

## 9. Training / Evaluation

Temporal splitting still applies to nuisance and outcome forecasts. Evaluate factual outcome prediction, propensity calibration/overlap, covariate balance after weighting, and stability across specifications. Causal effects cannot generally be validated against unobserved counterfactuals; use randomized holdouts when possible, pre-period fit, placebo intervention dates/outcomes/units, negative controls, sensitivity to hidden confounding, and confidence intervals accounting for time dependence. Policy evaluation should report expected utility, risk, and constraints—not only ATE.

## 10. Complexity and Cost

Cost depends on nuisance forecasters and design. Simple regression/IPW is linear-ish in rows/features per fit; cross-fitting multiplies training by fold count. Synthetic control solves constrained optimization over donor units. Bayesian structural time-series sampling can be expensive. Sequential g-computation may simulate many trajectories; deep causal models need GPUs only when scale/complexity warrants them.

## 11. Common Use Cases

- Forecast demand/revenue under prices, promotions, and campaigns.
- Estimate policy, product launch, outage, or intervention impact.
- Optimize treatment schedules in healthcare.
- Plan capacity when allocation decisions affect demand.
- Counterfactual incident analysis and causal anomaly attribution.

## 12. Common Mistakes

- Calling a temporally masked forecast “causal forecasting” without interventions.
- Interpreting feature importance/attention/correlation as causal effect.
- Conditioning on post-treatment mediators or colliders.
- Using future information, revised covariates, or intervention-contaminated controls.
- Ignoring time-varying confounding, anticipation, interference, and positivity.
- Choosing controls after viewing post-treatment outcomes.
- Treating good factual RMSE as proof of counterfactual validity.
- Reporting effect estimates without assumptions/sensitivity analysis.

## 13. Edge Cases / Limitations

Hidden confounding makes observational effects unidentified without additional assumptions/instruments. No overlap means the requested policy extrapolates beyond evidence. Spillovers violate independent-unit assumptions. One treated time series gives limited effective sample size; autocorrelation narrows naive standard errors incorrectly. Simultaneous interventions, changing treatment versions, feedback, and nonstationary mechanisms make counterfactual transport difficult.

## 14. Variations

| Variation                       | What changes / when useful                             | Importance            |
|---------------------------------|--------------------------------------------------------|-----------------------|
| Interrupted time series         | Models level/slope change at intervention              | Placement/project     |
| Difference-in-differences       | Treated-control trend contrast                         | Core causal interview |
| Synthetic control               | Weighted donor counterfactual for one/few units        | Strong project        |
| Bayesian structural time series | Probabilistic counterfactual using controls/components | Advanced              |
| Marginal structural model       | IP weights for time-varying treatment/confounding      | Research/health       |
| Causal ML/uplift                | Heterogeneous effects for targeting                    | AI engineer roles     |
| Structural causal model         | Explicit mechanisms/interventions                      | Research              |

## 15. Related Topics

Ordinary forecasting estimates associations under the observed policy; causal inference changes the policy. Granger causality asks whether past **X** improves prediction of **Y**, not necessarily intervention causality. ARIMAX/intervention models can estimate effects only with valid design assumptions. Synthetic controls and BSTS combine time-series prediction with causal identification. Reinforcement learning targets sequential policies but needs experimental/off-policy assumptions.

## 16. Interview Questions

1. **Forecasting versus causal forecasting?** Predict observed-policy future versus potential future under a defined intervention.
2. **What is a counterfactual?** The outcome that would occur under an action different from the observed one.
3. **Why can prediction succeed but causality fail?** Correlated proxies predict well without representing intervention effects.
4. **Core identification assumptions?** Consistency, exchangeability/no unmeasured confounding, positivity; plus no harmful interference as relevant.
5. **What is time-varying confounding?** A confounder evolves, predicts later treatment/outcome, and may itself be affected by earlier treatment.
6. **Why not adjust for every variable?** Mediators remove part of effect; colliders introduce bias; post-treatment values leak consequences.
7. **What is doubly robust?** Consistency if either outcome or propensity nuisance model is correctly specified, under causal assumptions.
8. **DiD key assumption?** In absence of treatment, treated and control outcomes would follow parallel trends.
9. **Is Granger causality causal?** It is predictive precedence; causal meaning requires stronger structural assumptions.
10. **How validate causal forecast?** Experiments when possible, otherwise design diagnostics, placebos, negative controls, sensitivity, and later policy outcomes.

## 17. Practice Tasks

- Draw a DAG for promotions targeted to low expected demand.
- Simulate confounding and compare naive regression, IPW, and AIPW.
- Implement DiD and test pre-intervention trends/placebo dates.
- Debug adjustment for a post-treatment mediator.
- Extend static treatment to a two-stage longitudinal g-computation experiment.

## 18. Project Ideas

| Project                    | Description                                         | Stack/data                                   | Resume value               |
|----------------------------|-----------------------------------------------------|----------------------------------------------|----------------------------|
| Campaign impact lab        | Synthetic control/BSTS counterfactual with placebos | pandas, CausalImpact/PyMC; CausalImpact data | Design + uncertainty       |
| Promotion policy simulator | Heterogeneous uplift and inventory-aware policy     | sklearn/EconML; retail/synthetic             | Decision-focused causal ML |
| Policy DiD study           | Staggered policy effect with pretrend diagnostics   | statsmodels; public regional panel           | Econometric rigor          |

## 19. Quick Revision

- **Key idea:** forecast potential outcomes under explicit interventions.
- **Formula:**  $ATE_h = E[Y_{\{t+h\}}(1) - Y_{\{t+h\}}(0)]$ .
- **Use:** when an action changes the future; **metrics:** factual forecast, balance/overlap, placebo/sensitivity, policy value.
- **Trap:** prediction  $\neq$  intervention effect; post-treatment adjustment and hidden confounding.
- **One-liner:** "Causal forecasting needs an estimand and identification design before it needs a forecasting model."

## 20. Final Cheat Sheet

| Item            | Summary                                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------|
| Definition      | Potential-outcome forecast under a specified intervention                                                                        |
| Input/output    | Longitudinal outcomes/treatments/confounders $\rightarrow$ counterfactual paths/effects                                          |
| Main steps      | Define estimand $\rightarrow$ DAG/design $\rightarrow$ point-in-time data $\rightarrow$ estimate $\rightarrow$ falsify/sensitize |
| Hyperparameters | History/horizon, nuisance models, weights/clipping, donors/priors                                                                |
| Metrics         | Policy value, effect uncertainty, overlap/balance, placebo and calibration checks                                                |
| Pros/cons       | Answers decisions / relies on untestable identification and support                                                              |
| Best use cases  | Pricing, campaigns, policy, medicine, intervention planning                                                                      |

## Cross-Topic Interview Comparison

| Need                                       | Strong first candidates              | Why                                             |
|--------------------------------------------|--------------------------------------|-------------------------------------------------|
| Stable local level                         | Naive, moving average, SES           | Minimal variance and cost                       |
| Trend + one season                         | Holt-Winters, SARIMA, Prophet        | Explicit interpretable components               |
| Linear autocorrelation                     | ARIMA/SARIMA                         | Statistical diagnostics and intervals           |
| Business calendars/change points           | Prophet or regression + ARIMA errors | Holidays/Fourier/piecewise trend                |
| Many related nonlinear series              | LSTM, TCN, Transformer               | Global parameter sharing                        |
| Finite multiscale context, high throughput | TCN                                  | Parallel causal convolutions                    |
| Long content-dependent context, large data | Transformer                          | Global attention and flexible fusion            |
| Latent dynamics/noisy measurements         | State-space model + Kalman filter    | Recursive posterior uncertainty                 |
| Effect of an action                        | Causal forecasting design            | Counterfactual rather than correlational target |

## Final Placement Strategy

In an interview, begin with the operational definition: target, frequency, forecast origin, horizon, available covariates, and decision cost. Establish last-value and seasonal-naive baselines. Explain chronological rolling validation and leakage controls before proposing a model. Choose the smallest model that matches the structure, report point and probabilistic metrics by horizon, inspect residuals and failure slices, and distinguish predictive association from causal intervention effects.